B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institute, Affiliated to VTU, Belagavi)

Bull Temple Road, Basavanagudi, Bengaluru – 560019

A Mini-Project Report on

LoomTrace: An Agentic Verifier for Geographical Indication Claims on Indian Handloom Products

Submitted as the Alternative Assessment Tool (AAT) for the course

Agentic AI (26CD7PEAAI)

of

Bachelor of Engineering

in

Computer Science and Engineering (Data Science)

Submitted by (Team No. 10)

1BM23CD027	**Sudeeksha**
1BM23CD026	**Khushi Vernekar**
1BM23CD066	**Trishul J**
1BM24CD400	**Geetha R**

Course Lead

Dr. Kalyan Nagaraj

Assistant Professor, Department of CSE (Data Science)

B.M.S. College of Engineering, Bengaluru

Department of Computer Science and Engineering (Data Science)

B.M.S. College of Engineering, Bengaluru

Academic Year 2026–27

CERTIFICATE

B.M.S. College of Engineering, Bengaluru
Department of Computer Science and Engineering (Data Science)

This is to certify that the mini-project entitled **"LoomTrace: An Agentic Verifier for Geographical Indication Claims on Indian Handloom Products"** has been carried out by Sudeeksha (1BM23CD027), Khushi Vernekar (1BM23CD026), Trishul J (1BM23CD066) and Geetha R (1BM24CD400), bona fide students of B.M.S. College of Engineering, Bengaluru, as the Alternative Assessment Tool (AAT) for the evaluation of the course **Agentic AI (26CD7PEAAI)**, Semester VII, Bachelor of Engineering in Computer Science and Engineering (Data Science), during the academic year 2026–27. The work has been evaluated through the four project reviews prescribed for the course, and this report satisfies the requirements of the AAT component of the said course.

Dr. Kalyan Nagaraj
Course Lead, Agentic AI (26CD7PEAAI)
Assistant Professor, Dept. of CSE (Data Science)
B.M.S. College of Engineering, Bengaluru

Date: _____

ABSTRACT

Handloom products in India are frequently sold with claims of a Geographical Indication (GI) tag, such as Kancheepuram Silk or Pochampalli Ikat, but a buyer or marketplace has no quick way to check whether such a claim is backed by official records. The evidence needed is spread across the GI registry, the GI application documents, the national handloom census and producer records, and a single prompt to a language model can neither consult these sources nor guarantee that its answer is grounded in them. This report presents LoomTrace, an agent that checks a GI claim on a handloom product and returns an advisory, cited verdict: Supported, Conflicting evidence, or Insufficient evidence, the last of which is routed to a human reviewer. LoomTrace is implemented as a LangGraph state graph of six nodes (extraction, planning, a retriever agent, a verifier agent, an evidence grader and a guardrail) with a bounded re-planning loop, seven typed tools, keyword-based retrieval over official GI text, per-node checkpoints in an SQLite database, and a web interface with a review queue. Reference data cover fifteen GI-registered handloom products and the state-wise weaver counts of the Fourth All India Handloom Census 2019–20; individual weaver and production records are simulated. On a labelled set of 45 cases the agent reached 100% verdict accuracy and correctly routed all 15 unclear cases to human review, while a single-lookup baseline reached 33.3% and never abstained. Disabling the re-planning loop reduced accuracy to 80.0%, at a mean saving of 3.1 milliseconds per case. The main limitation is that the test set was constructed by the team from simulated producer records, so these results demonstrate correct behaviour on controlled cases rather than accuracy on real marketplace listings.

Keywords: agentic AI, geographical indications, handloom, retrieval-augmented generation, human-in-the-loop, LangGraph.

CONTENTS

Certificate ii

Abstract iii

List of Figures vii

List of Tables viii

List of Abbreviations ix

1 Introduction 1

1.1 Background . . . 1
1.2 Problem Statement . . . 2
1.3 Motivation and Why Agentic AI . . . 2
1.4 Objectives . . . 3
1.5 Scope of the Project . . . 4
1.6 Organisation of the Report . . . 4

2 Literature Survey and Existing Practice 5

2.1 Existing Practice in the Sector . . . 5
2.1.1 Legal basis of a GI claim . . . 5
2.1.2 How a claim is checked today . . . 5
2.1.3 Where current practice fails . . . 6
2.2 Related Technical Work . . . 6
2.2.1 Reasoning and Acting with Tools . . . 6
2.2.2 Retrieval-Augmented Generation . . . 7
2.2.3 Planning, Reflection and Multi-Agent Collaboration . . . 7
2.2.4 Evaluation of LLM-Based Systems . . . 7
2.3 Comparative Summary . . . 8
2.4 Research Gap . . . 9

3 Requirements and System Analysis 10

3.1 Stakeholders and Users . . . 10
3.2 Functional Requirements . . . 10
3.3 Non-Functional Requirements . . . 11
3.4 Data Sources . . . 12
3.5 Hardware and Software Requirements . . . 13

3.6 Feasibility . . . 13

4 Agent Design and Architecture 15
4.1 System Architecture . . . 15
4.2 Component Description . . . 16
4.2.1 Product Claim Input and Extract Node . . . 16
4.2.2 Planner . . . 16
4.2.3 Retriever Agent and Tools . . . 17
4.2.4 Verifier Agent . . . 17
4.2.5 Grader . . . 17
4.2.6 Guardrail and Human Review Gate . . . 18
4.2.7 Review Queue and Human Reviewer . . . 18
4.2.8 Data Stores and Checkpoints . . . 18
4.3 Selected Agentic Patterns and Justification . . . 18
4.4 Control Flow and Agent State . . . 19
4.5 Data Flow . . . 21

5 Agentic Concepts Demonstrated 22
5.1 Concept Coverage Matrix . . . 22
5.2 Evidence for Each Concept . . . 23
5.2.1 Tool Use . . . 23
5.2.2 Agentic RAG . . . 24
5.2.3 Planning and Reasoning . . . 24
5.2.4 Observability . . . 25
5.2.5 Evaluation . . . 25
5.2.6 Bonus Concepts . . . 25

6 Implementation 28
6.1 Technology Stack . . . 28
6.2 Data Preparation and Indexing . . . 29
6.3 Core Agent Loop . . . 30
6.4 Tools and Function Definitions . . . 31
6.5 Prompt Design . . . 31
6.6 Guardrails and Loop Bounds . . . 32
6.7 User Interface . . . 32
6.8 Repository Structure . . . 34

7 Observability and Evaluation 35
7.1 Tracing and Monitoring . . . 35
7.2 Evaluation Dataset . . . 36

7.3 Metrics and Evaluation Rubric . . . 36
7.4 Baselines . . . 37
7.5 Results . . . 37
7.6 Error Analysis . . . 40

8 Experimentation **42**
8.1 Hypothesis . . . 42
8.2 Experimental Setup . . . 42
8.3 Results . . . 43
8.4 Discussion and Decision . . . 44

9 Limitations, Ethics and Safety **45**
9.1 Limitations . . . 45
9.2 Failure Modes and Guardrails . . . 46
9.3 Ethical and Responsible-Use Considerations . . . 47

10 Conclusion and Future Work **48**
10.1 Conclusion . . . 48
10.2 Future Work . . . 49

References **50**

A Team Contribution **53**

B Setup and Run Instructions **54**

C Sample Traces and Screenshots **55**
C.1 Successful Run with Correction: Kanjivaram Silk Saree . . . 55
C.2 Contradiction: Banarasi Saree Claimed from Surat . . . 55
C.3 Abstention: Banaras Saree without Producer Details . . . 56

D Prompts **58**

LIST OF FIGURES

4.1 Architecture of LoomTrace: six-node state graph, tools, data stores and human review 16

5.1 Reviewer dashboard: open case with extracted claim, escalation reason, note field and decision buttons; below it, an earlier case already decided by a reviewer 26

5.2 Requester's view of an escalated case after the reviewer's decision, with the reviewer's note, the automated verdict and the progress of the case 27

6.1 Verification reports produced by the interface for two claims 33

7.1 Per-class F1-score on the 45-case golden set 39

8.1 Verdict accuracy (left) and mean tool calls per case (right) for the two variants 43

C.1 Report for a claim without weaver or batch details, escalated to human review 57

LIST OF TABLES

2.1 Comparison of related approaches with the proposed system 8

3.1 Data sources used in LoomTrace 12

4.1 Agentic patterns and design justification 19
4.2 Fields of the shared CaseState 20

5.1 Coverage of the agentic-concept checklist 23

6.1 Technology stack 28
6.2 Tools available to the agent 31

7.1 Evaluation results on the 45-case golden set (mean of 3 repetitions) . 38
7.2 Confusion matrices (rows: label; columns: prediction; S = Supported, C = Conflicting, I = Insufficient) 39
7.3 Results against the targets of the design document 39
7.4 Behaviour on six realistic inputs not in the golden set 41

8.1 A/B experiment: single planning pass (A) versus re-planning loop (B), 45 cases, 3 repetitions 43

9.1 Failure modes and mitigations 46

A.1 Contribution of team members 53

LIST OF ABBREVIATIONS

API	Application Programming Interface
BM25	Best Matching 25 (Okapi ranking function)
GI	Geographical Indication
HITL	Human-in-the-Loop
IP	Intellectual Property
JSON	JavaScript Object Notation
LLM	Large Language Model
OCR	Optical Character Recognition
PIB	Press Information Bureau
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
SLM	Small Language Model
TRIFED	Tribal Co-operative Marketing Development Federation of India
TRIPS	Trade-Related Aspects of Intellectual Property Rights
UI	User Interface

CHAPTER 1

INTRODUCTION

This chapter introduces the handloom sector and the problem of verifying Geographical Indication (GI) claims on handloom products. It states the decision that the proposed agent supports, explains why an agentic design is needed rather than a script or a single language-model prompt, and sets out the objectives and scope of the project.

1.1 Background

A Geographical Indication is a sign used on goods that have a specific geographical origin and that possess qualities, a reputation or characteristics essentially attributable to that origin. The Agreement on Trade-Related Aspects of Intellectual Property Rights (TRIPS) requires member countries to provide legal means to prevent the use of a geographical indication on goods that do not originate in the indicated area [17]. In India this obligation is implemented by the Geographical Indications of Goods (Registration and Protection) Act, 1999, under which a GI is registered by an association of producers, and individual producers may in turn be registered as authorised users [16]. Registrations are maintained by the Geographical Indications Registry of the Office of the Controller General of Patents, Designs and Trade Marks, commonly referred to as IP India [18].

Handloom textiles form one of the largest groups of registered Indian GIs. Products such as Kancheepuram Silk from Tamil Nadu, Pochampalli Ikat from Telangana, Banaras Brocades and Sarees from Uttar Pradesh, Sambalpuri Bandha from Odisha and Chanderi Sarees from Madhya Pradesh are each tied to named districts, weaving communities, materials and techniques [18, 20]. The sector is large and dispersed: according to the Fourth All India Handloom Census 2019–20, India has 26,73,891 handloom weavers and 8,48,621 allied workers, with the largest weaver populations in Assam (11,07,428), West Bengal (3,66,656) and Manipur (2,12,481) [19]. For the three products on which this project was first scoped, the census records 1,97,818 handloom weavers in Tamil Nadu, 25,930 in Telangana and 1,30,778 in Uttar Pradesh [19].

Because a GI signals both origin and traditional method, a GI claim adds commercial value to a product, and it is used prominently in product descriptions, labels and online listings. The Ministry of Textiles operates certification schemes such as the Handloom Mark and the India Handloom Brand to help buyers identify genuine handloom goods [21]. However, a buyer or a marketplace operator who sees the claim

"GI-tagged Kanjivaram silk saree" on a listing still has to establish several separate facts before trusting it: that the product name corresponds to a registered GI, that the stated place of manufacture lies within the registered geographical area, that the materials are consistent with the registered description, and that the weaver and production batch can be traced to the relevant weaving cluster.

1.2 Problem Statement

A buyer, marketplace moderator or cooperative officer who receives a handloom product with a GI-tag claim has no single place to check whether the claim is supported. The necessary evidence is spread across the GI registration list, the statement of case in each GI application, the national handloom census and producer-level weaver and batch records, and the product label itself may use a local name ("Kanjivaram" instead of "Kancheepuram Silk"), omit identifiers, or contain details that contradict the registration. Checking these sources manually is slow and inconsistent, and an unsupported claim may be accepted while a genuine product with incomplete paperwork may be wrongly rejected. **The decision supported by this project is: is this handloom product's GI-tag claim supported by the available official evidence, contradicted by it, or not decidable without a human reviewer?**

1.3 Motivation and Why Agentic AI

Three simpler alternatives were considered before an agentic design was adopted.

A fixed rule script. A script could compare a structured form against a lookup table. However, product labels are free text, products are referred to by local names and spellings ("Pochampally", "Ikkat", "Banarasi"), and the set of checks that is meaningful depends on which fields are present: a label without a batch number cannot be checked against provenance records, and a product that is registered for a whole state (for example, Muga Silk of Assam) must be checked differently from one registered for specific districts. A fixed pipeline either fails on such inputs or silently skips checks.

A single language-model prompt. A large language model (LLM) asked "is this saree authentic?" will answer from its training data. It cannot consult the current registry, it cannot query producer records, and its output cannot be traced to a source. Language models are known to produce fluent but unsupported statements [12], and smaller models are particularly weak when relevant facts are buried

in long contexts [13]. For a decision that affects a producer's reputation, an answer without citations is not acceptable.

Plain retrieval followed by one answer. A single document lookup followed by one decision improves grounding, but it cannot notice that its first search failed (for example, because the label used an alias), and it has no mechanism to stop and ask a human when evidence is missing. Chapter 7 shows that such a baseline marks every case in the test set as "Supported".

The problem therefore requires several capabilities that together motivate an agentic design. It requires *tool use*, because facts must come from registry, census and producer records rather than from model memory [3]. It requires *retrieval over official documents*, so that every verdict can cite the registered description [2]. It requires *planning and re-planning*, because the checks needed differ by claim and a weak first search must be detected and repeated with a better query [6]. It requires *evidence grading (reflection)* before a verdict is issued, so that the agent does not guess when evidence is incomplete. Finally, it requires *human-in-the-loop (HITL) escalation*, because abstaining and referring a case to a reviewer is safer than wrongly certifying or wrongly rejecting a product.

1.4 Objectives

The objectives of the project are as follows.

1. To design an agent, implemented as a state graph, that verifies a handloom GI claim against official GI and census data and producer records, and returns one of three advisory verdicts with citations.
2. To implement seven typed tools and retrieval over official GI text, with a bounded re-planning loop (at most three iterations) that refines the search when the first attempt is weak.
3. To implement a guardrail and a human review workflow so that every case without sufficient, cited evidence is routed to a reviewer, whose decision is recorded and visible to the requester.
4. To record a step-level trace and a persistent checkpoint for every node of every run.
5. To evaluate the agent on a labelled set of 45 cases against a single-lookup baseline, with targets of at least 80% verdict accuracy and at least 90% correct abstention.

6. To measure, through a controlled A/B experiment, the effect of the re-planning loop on accuracy, latency and tool usage.

1.5 Scope of the Project

Included. The system verifies GI-tag claims for fifteen registered handloom GIs: the three products named in the project's design document (Kancheepuram Silk, Pochampalli Ikat, and Banaras Brocades and Sarees) and twelve further handloom GIs whose geographical area and materials are stated in a Government of India GI catalogue [20]. Input is a product label or seller claim containing some or all of the following fields: product name, region, state, material, weaver identifier and batch identifier. Output is an advisory verdict with the list of checks performed and the sources cited. The system includes a web interface for requesters, a reviewer dashboard, and an offline evaluation harness.

Excluded. The system does not issue legal certification and does not replace the GI Registry or the authorised-user process under the GI Act [16]. It does not physically inspect textiles. It does not access live government portals at run time; reference data are stored locally. Individual weaver and batch records are not publicly available, and the records used are simulated, as stated in Section 3.4. Products outside the fifteen GIs are reported as "not covered" and may be sent to a reviewer, but they are not verified automatically. Label images are supported only when the optional optical character recognition (OCR) package is installed, and this path was not part of the evaluation.

1.6 Organisation of the Report

Chapter 2 reviews how GI claims are verified in practice and the technical literature on tool use, retrieval, planning, reflection and evaluation. Chapter 3 states the stakeholders, requirements and data sources. Chapter 4 presents the architecture of LoomTrace, its components, the agentic patterns chosen and the control and data flow. Chapter 5 maps the system to the agentic concepts of the course with evidence from real runs. Chapter 6 describes the implementation, including the technology stack, the agent loop, the tools, the guardrails and the user interface. Chapter 7 describes tracing and reports the evaluation against a baseline, and Chapter 8 reports the A/B experiment on the re-planning loop. Chapter 9 discusses limitations, failure modes and ethical considerations, and Chapter 10 concludes with future work.

CHAPTER 2

LITERATURE SURVEY AND EXISTING PRACTICE

This chapter first describes how GI claims on handloom products are established and checked in practice, and then reviews the technical literature on the agentic techniques used in LoomTrace. It ends with a comparative summary and a statement of the gap that the project addresses.

2.1 Existing Practice in the Sector

2.1.1 Legal basis of a GI claim

Under the Geographical Indications of Goods (Registration and Protection) Act, 1999, a GI is registered for a class of goods and a defined geographical area on the application of an association of persons or producers. The application contains a *statement of case* that describes the product, the geographical area, the method of production, the materials and the link between the product and its origin [16]. After registration, producers within the area may apply to be registered as *authorised users*, and only the registered proprietor and authorised users have the right to use the indication. The registry and its records are maintained by IP India [18]. The international basis for this protection is Section 3 of the TRIPS Agreement, which defines geographical indications and obliges members to prevent misleading use of them [17].

2.1.2 How a claim is checked today

In practice, the verification of a GI claim at the point of sale is manual and fragmented. A buyer in a shop or on an online marketplace sees a product description or label. To check the claim, the buyer or a marketplace moderator must find the corresponding GI registration and its application number, read the statement of case to learn the registered area and materials, and compare these with the place and materials stated by the seller. Where producer details are provided, they must be checked against cooperative or weaver records, which are not published centrally. The Ministry of Textiles supports buyer confidence through separate labelling schemes, notably the Handloom Mark, which certifies that a product is handwoven, and the India Handloom Brand, which certifies quality and process standards [21]. These labels are useful

signals, but they do not certify that a product falls within a particular GI, and they are not linked to the GI registry in a way that a buyer can query.

Government publications provide some of the reference data needed for verification. The TRIFED GI catalogue published through the Press Information Bureau (PIB) lists registered GI products marketed by Tribes India together with short descriptions of their place of production and materials [20]. The state-wise results of the Fourth All India Handloom Census 2019–20, published as a PIB annexure, give the number of handloom weavers and allied workers in each state and union territory [19]. These sources are authoritative but are distributed as separate PDF documents with different structures, and none of them is designed to answer the question "is this particular claim supported?".

2.1.3 Where current practice fails

Three weaknesses of current practice motivate this project. First, the evidence is spread across several documents that must be read together, so a complete check is slow and is rarely performed for an individual purchase. Second, product names on labels and listings frequently differ from the registered names, which makes simple string search unreliable. Third, when information is missing, there is no defined route for escalation: the checker must either accept or reject the claim without evidence, or abandon the check. A system that collects the evidence, explains which checks passed and failed, and routes undecidable cases to a person addresses all three weaknesses.

2.2 Related Technical Work

The technical literature relevant to LoomTrace is organised below into four themes: reasoning and acting with tools; retrieval-augmented generation; planning, reflection and multi-agent collaboration; and the evaluation of systems built on language models.

2.2.1 Reasoning and Acting with Tools

Yao *et al.* [1] proposed ReAct, in which a language model interleaves free-text reasoning steps with actions such as search queries, so that it can gather information before answering. Schick *et al.* [3] showed that a language model can learn to decide when to call external tools such as calculators and search engines, and how to use their results. Both works establish that grounding a model's output in tool results reduces reliance on memorised knowledge. LoomTrace adopts the tool-use principle, since every fact used in a verdict is returned by a typed tool with a source identifier. It does not, however, adopt free-form interleaving of thought and action: the sequence of nodes is fixed by an explicit graph, so that every run can be traced, bounded and check-

pointed. This trade-off favours auditability over flexibility, which suits a verification task whose steps are known in advance.

2.2.2 Retrieval-Augmented Generation

Lewis *et al.* [2] introduced retrieval-augmented generation (RAG), which conditions a generator on passages retrieved from an external corpus, and showed gains on knowledge-intensive tasks. Gao *et al.* [14] survey later developments and distinguish naive RAG, which retrieves once, from advanced and modular variants that rewrite queries, re-rank results and retrieve iteratively. Two adaptive variants are particularly relevant. Self-RAG, proposed by Asai *et al.* [5], trains a model to decide when to retrieve and to critique its own output with reflection tokens. Corrective RAG, proposed by Yan *et al.* [6], adds a lightweight evaluator that grades retrieved documents and triggers corrective actions, such as a new search, when retrieval is weak. LoomTrace follows the corrective pattern: its grader detects when the registry lookup or the document search did not resolve the product, and the planner then re-queries with the canonical product name and a filter on that product. For ranking, LoomTrace uses the BM25 function from the probabilistic relevance framework of Robertson and Zaragoza [11], which is fast, transparent and effective on short, keyword-rich official passages.

2.2.3 Planning, Reflection and Multi-Agent Collaboration

Wang *et al.* [10] proposed plan-and-solve prompting, in which a model first devises a plan that divides a task into subtasks and then executes it, which reduces missed steps compared with direct reasoning. Shinn *et al.* [4] proposed Reflexion, in which an agent evaluates its own attempt and uses verbal feedback to improve the next attempt. Wu *et al.* [7] presented AutoGen, a framework in which several specialised agents converse to complete a task. A survey by Wang *et al.* [8] organises agents built on language models by their profile, memory, planning and action modules. LoomTrace draws on each of these ideas in a constrained form. A planner decides which checks a claim requires; a grader evaluates the evidence before any verdict is produced and can send control back to the planner; and the gathering of evidence and the judging of evidence are assigned to two separate nodes, a retriever agent and a verifier agent, with the graph acting as the supervisor that routes between them.

2.2.4 Evaluation of LLM-Based Systems

Zheng *et al.* [9] studied the use of strong language models as judges of other models' outputs and reported high agreement with human preferences, while documenting position, verbosity and self-enhancement biases. Ji *et al.* [12] survey hallucination in

natural language generation and emphasise that fluency is not evidence of faithfulness. Liu *et al.* [13] show that models use information at the beginning and end of long contexts better than information in the middle. For LoomTrace, these findings have two consequences. First, because each verdict is one of three discrete labels and each check has a deterministic expected outcome, the system is evaluated with exact-match metrics against hand-assigned labels rather than with a model judge. Second, the design keeps every model call narrow and short and assigns the factual checks to deterministic rules, which is consistent with the known weaknesses of small models on long inputs.

2.3 Comparative Summary

Table 2.1 compares the most relevant approaches with LoomTrace on four properties: whether the approach calls external tools, whether retrieval is adaptive (graded and repeated when weak), whether it includes a human escalation path, and whether every decision is accompanied by citations to sources.

Table 2.1: Comparison of related approaches with the proposed system

Work	**Key idea**	**Tools**	**Adapt. RAG**	**HITL**	**Cited**
ReAct [1]	Interleaved reasoning and acting	✓	–	–	–
Toolformer [3]	Self-taught tool calling	✓	–	–	–
RAG [2]	Retrieval-conditioned generation	–	–	–	✓
Self-RAG [5]	Learned retrieval and self-critique	–	✓	–	✓
CRAG [6]	Retrieval grading and correction	–	✓	–	✓
Reflexion [4]	Verbal self-feedback across attempts	✓	–	–	–
Manual GI check [16, 18]	Person reads registry and records	–	–	✓	✓
LoomTrace (ours)	Graph agent with graded re-query and review queue	✓	✓	✓	✓

2.4 Research Gap

As Table 2.1 shows, the general agent techniques reviewed provide tool use, adaptive retrieval or self-critique individually, but they are not applied to GI verification, and none of them defines an escalation path to a human reviewer. Manual verification provides human judgement and citations but does not scale and has no structured way of handling missing information. No reviewed approach combines retrieval over official GI text, structured checks against registry, census and producer records, a bounded re-planning loop, and a human review workflow in a single traceable system for handloom GI claims; LoomTrace is designed to address this gap.

CHAPTER 3

REQUIREMENTS AND SYSTEM ANALYSIS

This chapter translates the problem stated in Chapter 1 into concrete requirements. Each functional requirement is numbered so that it can be traced to the design in Chapter 4 and tested in Chapter 7, and each non-functional requirement is given a numerical bound.

3.1 Stakeholders and Users

Four groups interact with or are affected by LoomTrace.

- **Requesters (primary users).** Buyers of handloom products, marketplace moderators who review seller listings, and retail staff who receive stock described as GI-tagged. They need a quick, understandable answer with the reasons and sources behind it, and a reference number with which to follow up.
- **Reviewers.** Staff of a weavers' cooperative society, a handloom development office or a marketplace trust-and-safety team who resolve cases that the agent cannot decide. They need the extracted claim, the reason for escalation and a way to record a decision with a note.
- **Weavers and GI proprietors (affected parties).** Producers within the registered area benefit when false claims are detected, but are harmed if a genuine product is wrongly marked as conflicting. Their interest is the main reason that the system abstains rather than guesses.
- **Course evaluators and developers.** They need step-level traces, reproducible evaluation scripts and clear documentation of what is real and what is simulated.

3.2 Functional Requirements

FR1 The system shall accept a product claim as free label text containing any subset of the fields product, region, state, material, weaver identifier and batch identifier, and optionally a label image.

FR2 The system shall extract these fields into a typed structure and resolve local product names and aliases to the registered GI name.

FR3 The system shall decide which checks are applicable to the claim and run them using tools over the GI registry, GI document text, census data, weaver records and batch records.

FR4 The system shall output exactly one of three verdicts, *Supported*, *Conflicting evidence* or *Insufficient evidence*, together with the result of every check and the source identifiers that support it.

FR5 The system shall re-plan and retrieve again when the product cannot be resolved or the retrieved GI text is weak, for at most three iterations in total.

FR6 The system shall send every *Insufficient evidence* verdict, and any verdict without a citation or below a confidence threshold, to a human review queue, and shall allow a requester to ask for review of any verdict.

FR7 The system shall allow a reviewer to record a final decision with a note, and shall allow the requester to see that decision using the case reference.

FR8 The system shall store the full case state after every node, and keep a history of all cases and decisions.

FR9 The system shall report claims for products outside the covered GIs as "not covered" rather than as conflicting.

3.3 Non-Functional Requirements

NFR1 **Latency.** A verdict shall be returned in under 15 seconds per case, the target set in the project's design document.

NFR2 **Bounded cost.** The agent shall execute at most three planning iterations per case, and therefore at most fifteen tool calls; in rule mode it shall make no paid API calls.

NFR3 **Traceability.** Every node execution shall be recorded with its inputs and outputs, and every verdict shall list its source identifiers.

NFR4 **Safety.** No *Supported* or *Conflicting* verdict shall be issued without at least one citation and a grade of at least 0.6; otherwise the agent shall abstain.

NFR5 **Quality.** Verdict accuracy on the labelled test set shall be at least 80%, and at least 90% of cases labelled *Insufficient evidence* shall be routed to human review.

NFR6 Reproducibility. All reported numbers shall be reproducible by running the scripts in the repository on a laptop without internet access.

3.4 Data Sources

Table 3.1 lists the data sources used. Two of them are official Government of India publications, one is the IP India registration information recorded in the project's design document, and the producer-level records are simulated because individual weaver and batch data are not published.

Table 3.1: Data sources used in LoomTrace

Source	Description and use	Size	Origin
GI registry entries (3 products)	GI application number, state, districts and materials for Kancheepuram Silk (GI 15), Pochampalli Ikat (GI 4) and Banaras Brocades and Sarees (GI 99); registry checks	3 records	IP India registration list [18], as recorded in the design document
GI catalogue (12 products)	Place of production, materials and description of 12 further handloom GIs; registry checks and retrieval corpus	12 records, 12 passages	PIB / TRIFED, Nov. 2021 [20]
GI description passages (3 products)	Short summaries of the registered area, method and materials; retrieval corpus	9 passages	Written by the team from the GI descriptions; page numbers indicative
Handloom census	Handloom weavers and allied workers per state/UT; plausibility check	31 rows	PIB annexure, release ID 2202869 [19]
Weaver registry	Weaver identifier, name, cluster, district, looms	180 records	**Simulated** (seeded generator)
Production batches	Batch identifier, weaver, loom, yarn, start and finish dates	360 records	**Simulated** (seeded generator)
Golden test set	Labelled claims, 3 products × 3 verdicts × 5	45 cases	Constructed by the team
Demonstration tests	Hand-written claims covering 9 products	15 cases	Constructed by the team

The government data were obtained by downloading the two PIB PDF documents

and extracting their text. The census table was transcribed in full, including all 31 states and union territories. From the GI catalogue, only handloom textile entries whose place of production is stated in the document were used; two handloom entries, Khandua Saree and Karvath Kati Sarees, were excluded because the catalogue does not name their area of production. The catalogue does not print GI application numbers, so for these twelve products the system reports "Registered GI" without a number rather than supplying a number from an unverified source.

The weaver and batch records are generated by a seeded pseudo-random generator (seed 42), so the same records are produced on every run. For each product, 12 weavers are assigned in rotation to the districts of that product's GI area, and each weaver is given two batches whose yarn is drawn from the product's registered materials. These records are labelled as simulated in the source code, in the tool outputs ("SIMULATED record") and in the user interface.

3.5 Hardware and Software Requirements

Development and all evaluation runs used an Apple MacBook with an M2 processor and 8 GB of memory, running macOS 15.6. The software is written in Python 3.13 and uses LangGraph for orchestration, Pydantic for typed tool schemas, SQLite for persistence, Streamlit for the interface and pandas for tabular display. The optional language-model path uses the Groq API with the Llama 3.1 8B Instant model, and the optional OCR path uses EasyOCR. Exact versions are given in Table 6.1 and Appendix B. No graphics processor is required.

3.6 Feasibility

Technical feasibility. All checks reduce to lookups and comparisons over small tables and a corpus of 21 passages, so they can be executed in milliseconds on a laptop. The only components that require external services, the language model and OCR, are optional and are isolated behind tool interfaces with deterministic fall-backs.

Cost. All results in this report were produced in rule mode, without an API key, and therefore made zero language-model calls and incurred no cost. In the optional model mode, the design limits model usage to two short calls per case (field extraction and verdict explanation); the model selected is available on the free tier of the Groq service [25].

Time and scope control. The scope was limited to verification against stored reference data; live access to government portals was avoided because the IP India

application-detail pages returned errors during data collection and because the evaluation must be reproducible offline. The project was initially scoped to three products, as specified in the design document, and was extended to fifteen products once the official GI catalogue had been obtained.

CHAPTER 4

AGENT DESIGN AND ARCHITECTURE

This chapter presents the architecture of LoomTrace, describes each component shown in the architecture diagram, justifies the agentic patterns that were selected against the alternatives that were rejected, and describes the control flow, the shared state and the flow of data through the system for one concrete input.

4.1 System Architecture

Figure 4.1 shows the architecture of LoomTrace. A product claim enters at the top left and passes through six nodes of a state graph: *Extract*, *Planner*, the *Retriever agent*, the *Verifier agent*, the *Grader* and the *Guardrail*. The retriever agent calls typed tools that read four data stores. The grader decides whether the evidence is sufficient; if it is not, control returns to the planner, at most three times in total. The guardrail issues the verdict with its citations, or escalates the case to a review queue, from which a human reviewer records the final decision. All nodes read and write a single shared state object, `CaseState`, which is saved to an SQLite database after every node.

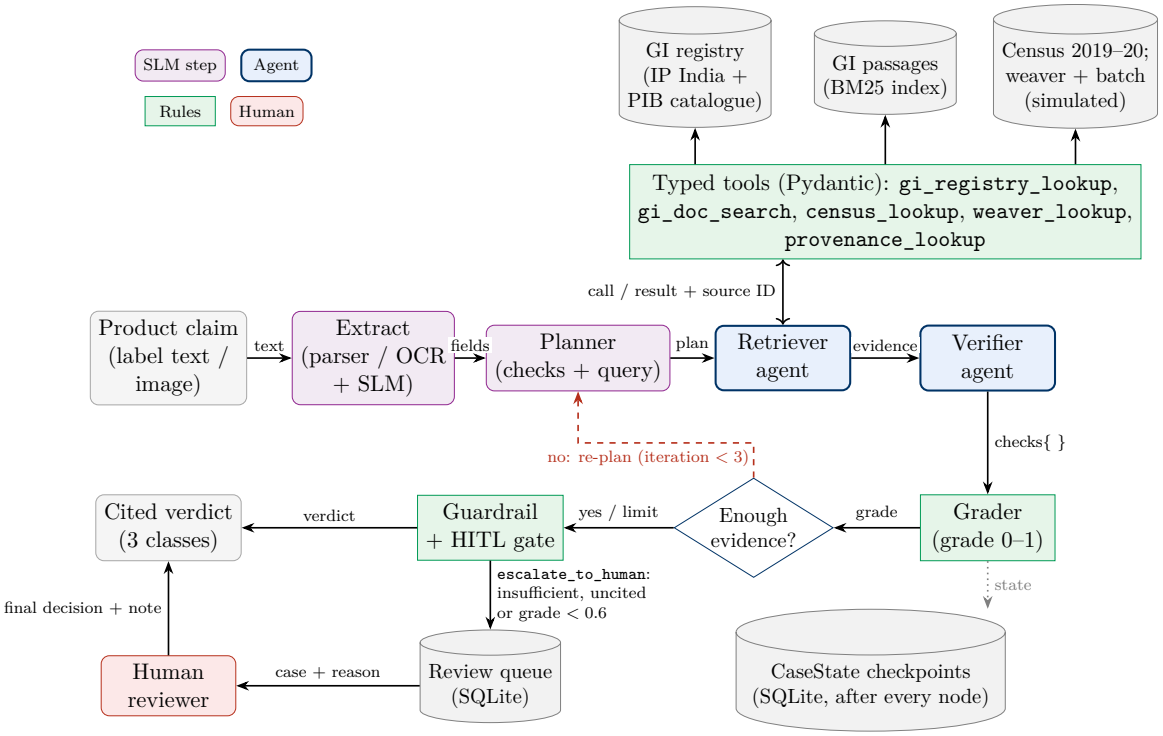


Figure 4.1: Architecture of LoomTrace: six-node state graph, tools, data stores and human review

4.2 Component Description

Each component in Figure 4.1 is described below in terms of its responsibility, its inputs and outputs, the method it uses, and its behaviour on failure.

4.2.1 Product Claim Input and Extract Node

The input is the text of a product label or seller claim, optionally accompanied by a label image. The *Extract* node calls the `extract_label` tool, which produces a typed `LabelFields` object with the fields `product`, `region`, `state`, `material`, `weaver_id`, `batch_id` and a Boolean `gi_claim`. If an image is provided and the EasyOCR package [26] is installed, the image text is recognised first. If an API key is configured, a small language model (SLM), Llama 3.1 8B [15], is asked to return the fields as JSON. Otherwise, and whenever the model call fails, a deterministic parser reads "key: value" lines, recognises identifier patterns such as `WV-KAN-001` and `B-KAN-0010`, and falls back to searching the text for known product names. Missing fields are left empty rather than guessed, so that later nodes can treat them as unknown.

4.2.2 Planner

The planner decides which checks the claim requires and what to search for. Three checks are always planned (product registered, GI document support and census plau-

sibility); region, state and material checks are planned only when the corresponding field is present; and four producer checks (weaver registered, weaver in the correct cluster, batch found, batch woven by the claimed weaver) are always planned so that their absence is visible. On the first iteration the planner requests an exact match in the registry and an unfiltered document search using the product name and region as the query. On a re-plan it resolves the product name to its registered form by matching the longest known alias, and builds a sharper query from the canonical name, region, state, registered materials and words that occur in statements of case ("geographical area weavers woven"), filtered to that product. If the product cannot be resolved, the second iteration records the product check as failed rather than looping indefinitely.

4.2.3 Retriever Agent and Tools

The retriever agent executes the plan by calling the tools listed in Table 6.2 in Chapter 6: `gi_registry_lookup`, `gi_doc_search`, `census_lookup` and, when the identifiers are present, `weaver_lookup` and `provenance_lookup`. Every tool returns a typed `ToolResult` containing a success flag, the data, a source identifier, a note and the elapsed time. The retriever appends each result to the evidence list in the state, tagged with the iteration number. Tool failures, such as an unknown weaver identifier, are returned as results with `ok=false` and an explanatory note; they are never raised as exceptions, so that the verifier can reason about them.

4.2.4 Verifier Agent

The verifier agent judges the evidence of the current iteration. It runs up to ten rule-based checks and assigns each a status of *pass*, *fail* or *unknown*, together with a plain-language reason and the source identifier on which the status rests. For example, the region check passes if the claimed region matches one of the registered districts, fails if it does not, and is unknown if the product could not be resolved; for GIs registered for a whole state, such as Muga Silk, the check compares the state instead. A claimed weaver who is not in the registry yields *unknown* for the batch–weaver match rather than *fail*, because a mismatch cannot be established against an unverified identity. Deterministic rules are used for these checks because they are exact and repeatable, which a small model is not.

4.2.5 Grader

The grader scores the evidence before any verdict is produced. It computes completeness C, the fraction of checks that are not unknown, and the citation fraction Q, the fraction of determinate checks that carry a source identifier. If no check has failed, the grade is $g = 0.7C + 0.3Q$; if a check has failed, the grade is $g = 0.5 + 0.5Q$, because

a cited contradiction is strong evidence even when other checks are incomplete. The grader then decides whether to loop: it requests a re-plan if the product check or the document-support check is unknown, a product name is present, and fewer than three iterations have been used. Otherwise it passes control to the guardrail.

4.2.6 Guardrail and Human Review Gate

The guardrail converts the checks into a verdict: *Conflicting evidence* if any check failed, *Insufficient evidence* if any check is unknown, and *Supported* only if every check passed. It then enforces two conditions: a decided verdict must have at least one citation, and its grade must be at least 0.6; if either condition fails, the verdict is downgraded to *Insufficient evidence*. Every *Insufficient evidence* verdict is passed to the `escalate_to_human` tool, which inserts the case, the extracted claim and the reason (for example, "missing: weaver_registered, batch_found") into the review queue. The guardrail also writes the case to the case history.

4.2.7 Review Queue and Human Reviewer

The review queue is a table in the SQLite database. A reviewer sees each open case with its details and reason, writes a note, and records one of three decisions (confirm authentic, not authentic, cannot confirm). The decision is stored in a separate table and becomes the final verdict for the case, and the requester can see it, with the note, by entering the case reference. A requester can also send a case that was decided automatically to the queue if they disagree with the result.

4.2.8 Data Stores and Checkpoints

The four data stores are the GI registry (fifteen products), the GI passage index (21 passages), the census table (31 rows) and the simulated weaver and batch records (180 and 360 records), described in Section 3.4. The `CaseState` checkpoint store is an SQLite table to which the full state is written after every node, giving each case its own memory and a durable history.

4.3 Selected Agentic Patterns and Justification

Table 4.1 lists the patterns used, where each is applied, why it was chosen and which alternative was rejected.

Table 4.1: Agentic patterns and design justification

Pattern	**Where**	**Reason**	**Alternative rejected**
Tool use	Retriever agent	Facts must come from registry, census and producer records, each with a source identifier	Answering from model memory
Agentic (corrective) RAG	Planner, retriever, grader	Labels use aliases; a weak first search must be detected and repeated with a sharper, filtered query [6]	Single-shot retrieval
Plan-and-execute with re-planning	Planner, grader loop	Applicable checks depend on which fields are present; steps are known, so they can be bounded and traced [10]	Free-form ReAct loop [1]
Supervisor multi-agent	Graph routes between retriever and verifier	One narrow responsibility per node: gathering evidence is separated from judging it	One node doing both
Reflection (evidence grading)	Grader	Verdicts must not be issued on incomplete evidence [4]	No self-check
HITL and guardrails	Guardrail, review queue	A wrong verdict harms a producer or a buyer; abstention is safer than a guess	Fully automatic decision

The main design decision was to use an explicit graph with deterministic checks rather than a free-form ReAct agent. In a free-form agent, the model chooses which tool to call next and when to stop, which makes the number of steps unbounded and the trace harder to audit. In LoomTrace the sequence of nodes is fixed, only the plan contents and the loop decision vary, and the loop has a hard limit. A second decision was to keep model calls narrow. Small models are fast and inexpensive but are weaker at long, open-ended reasoning [13]; giving the model only extraction and explanation, while rules perform the factual checks, keeps the system correct even when no model is available.

4.4 Control Flow and Agent State

Control moves through the graph as follows. The graph starts at *Extract*, then runs *Planner*, *Retriever*, *Verifier* and *Grader* in sequence. A conditional edge after the grader routes to the planner when the grader's decision is "re-plan" and to the

guardrail when it is "finish"; the guardrail leads to the end of the graph. The loop terminates under exactly three conditions: (i) the product and document checks are both determinate; (ii) no product name was extracted, so re-planning cannot help; or (iii) the iteration counter has reached the limit of three iterations. In addition, LangGraph's recursion limit is set to 40 node executions as a second, independent bound. A full run without re-planning executes 6 nodes, and each re-plan adds 4 nodes.

Table 4.2 lists the fields of the shared state.

Table 4.2: Fields of the shared CaseState

Field	Content	Written by
`case_id`	Unique reference, e.g. `CASE-94767A68`	Input
`label_text, image`	Raw claim text and optional image bytes	Input
`fields`	Extracted `LabelFields`	Extract
`plan, query, resolve_aliases`	Checks to run, search query, alias-resolution flag	Planner
`canonical`	Registered product name, once resolved	Retriever
`evidence[ ]`	Tool results with source IDs and iteration (append-only)	All tool-calling nodes
`checks{ }`	Status, reason and source per check	Verifier
`grade, grade_detail`	Score 0–1, completeness, consistency, decision	Grader
`iteration`	Planning iterations used (0–2)	Extract, Grader
`verdict, confidence, citations, rationale`	Final output	Guardrail
`escalated`	Review-queue entry, if any	Guardrail
`trace[ ]`	One entry per node with a summary message (append-only)	All nodes

The `evidence` and `trace` fields are declared with an additive reducer, so that each node appends to them instead of replacing them. This preserves the evidence of every iteration and makes the full history of a case available to the verifier, the interface and the checkpoint store.

4.5 Data Flow

The following example traces one real input through the system. The label reads "Product: Kanjivaram Silk Saree; Region: Thiruvallur; State: Tamil Nadu; Material: mulberry silk; Weaver ID: WV-KAN-001; Batch ID: B-KAN-0010".

1. **Extract** produces the six fields; the product is the alias "Kanjivaram Silk Saree".
2. **Planner (iteration 0)** plans ten checks and the query "Kanjivaram Silk Saree Thiruvallur", with exact registry matching.
3. **Retriever** calls five tools. The registry lookup fails (no exact match); the document search returns five passages, two from the Kancheepuram corpus and three from other products; the census, weaver and batch lookups succeed.
4. **Verifier** reports 5 pass, 0 fail and 5 unknown: the product, region, state, material and cluster checks cannot be made without a registry entry.
5. **Grader** computes completeness 0.5 and citation fraction 1.0, so $g = 0.65$; because the product check is unknown, it requests a re-plan.
6. **Planner (iteration 1)** resolves the alias to "Kancheepuram Silk" and issues a query filtered to that product.
7. **Retriever** now finds registry entry GI 15 and three Kancheepuram passages.
8. **Verifier** reports 10 pass, 0 fail, 0 unknown; **Grader** computes $g = 1.0$ and finishes.
9. **Guardrail** issues *Supported* with seven citations: three GI passages, the registry entry, the Tamil Nadu census row, and the weaver and batch records.

The case executed ten nodes, made eleven tool calls and wrote ten checkpoints. The complete trace of this run is given in Listing 5.2 in Chapter 5.

CHAPTER 5

AGENTIC CONCEPTS DEMONSTRATED

This chapter maps LoomTrace to the agentic-concept checklist of the course. All five mandatory concepts are implemented. Of the five bonus concepts, two are fully implemented and three are implemented in part; the partial ones are stated as such. Every item is supported by evidence from a real run: a trace excerpt, a tool log, a screenshot or a section of this report. All traces in this chapter were produced by the system on 11 November 2026 and are reproduced without editing, except that long lines are wrapped and non-ASCII symbols (tick, cross, arrow and middle dot) are replaced by the ASCII equivalents OK, X, `->` and a full stop.

5.1 Concept Coverage Matrix

Table 5.1 summarises the coverage.

Table 5.1: Coverage of the agentic-concept checklist

Tier	Concept	Status	Evidence
Mandatory	Tool / function use	✓	Seven typed tools; Listing 5.1, Table 6.2
Mandatory	Agentic RAG	✓	Weak first retrieval detected and repeated with a filtered query; Listings 5.2, 5.1
Mandatory	Planning and reasoning	✓	Planner output and re-plan in Listing 5.2; Algorithm 1
Mandatory	Observability / tracing	✓	Node trace and per-node SQLite checkpoints; Section 7.1, Listing 5.3
Mandatory	Evaluation	✓	45-case labelled set, baseline comparison; Chapter 7
Bonus	Multi-agent collaboration	Partial	Separate retriever and verifier nodes routed by the graph (supervisor); both are deterministic, not independent model agents
Bonus	Memory	Partial	Per-case memory in CaseState and checkpoint history; no memory across cases
Bonus	Human-in-the-loop and guardrails	✓	Guardrail, review queue, reviewer decisions; Listing 5.4, Figs. 5.1, 5.2
Bonus	Durable workflow	Partial	Full state persisted after every node; automatic resumption from a checkpoint is not implemented
Bonus	Experimentation	✓	A/B experiment on the re-planning loop; Chapter 8

5.2 Evidence for Each Concept

5.2.1 Tool Use

Listing 5.1 shows the tool log of the run described in Section 4.5, as stored in the `evidence` field of the state. Each line gives the planning iteration, the tool name, whether it succeeded, the source identifier it returned and its note. In iteration 0 the registry lookup fails because "Kanjivaram Silk Saree" is not the registered name, while the other tools succeed. In iteration 1 the same tools are called with the resolved product, and the registry returns entry GI 15. Simulated records are marked as such in the tool output itself.

Listing 5.1: Tool calls for the Kanjivaram case (iteration, tool, ok, source ID, note)

```
-  extract_label      True   LABEL#input                    parsed label text
0  gi_registry_lookup False  None   'Kanjivaram Silk Saree' not found (exact match)
```

```
0  gi_doc_search      True   GI15-p3,GI15-p5,PIB-GI-2021-p4,PIB-GI-2021-p7,
                             PIB-GI-2021-p12              5 passages (unfiltered)
0  census_lookup      True   HLCENSUS-2019-20#Tamil_Nadu
0  weaver_lookup      True   SIM-WEAVER#WV-KAN-001        SIMULATED record
0  provenance_lookup  True   SIM-PROV#B-KAN-0010          SIMULATED record
1  gi_registry_lookup True   IPINDIA-GI-REG#15
1  gi_doc_search      True   GI15-p3,GI15-p5,GI15-p7      3 passages filtered to
                                                          Kancheepuram Silk
1  census_lookup      True   HLCENSUS-2019-20#Tamil_Nadu
1  weaver_lookup      True   SIM-WEAVER#WV-KAN-001        SIMULATED record
1  provenance_lookup  True   SIM-PROV#B-KAN-0010          SIMULATED record
```

5.2.2 Agentic RAG

Retrieval in LoomTrace is agentic in the sense used by corrective RAG [6]: the result of retrieval is evaluated, and a poor result changes what the agent does next. In Listing 5.1, the unfiltered search in iteration 0 returns five passages, of which only two belong to Kancheepuram Silk; the other three describe other products. Because the registry did not resolve the product, the grader marks the evidence as insufficient. The planner then rewrites the query using the canonical name, the region, the state and the registered materials, and restricts the search to the resolved product; the second search returns three passages, all from the Kancheepuram corpus. The verdict cites these passages, so every *Supported* or *Conflicting* verdict is grounded in official text about the specific product.

5.2.3 Planning and Reasoning

Listing 5.2 shows the node-level trace of the same run. The planner first produces an initial plan of ten checks with exact matching. After the verifier finds five checks unknown, the grader computes a grade of 0.65 with completeness 0.5 and requests a re-plan. The second plan resolves the alias, after which all ten checks pass and the grader finishes with a grade of 1.0. The reasoning over the evidence is explicit in the verifier's per-check reasons, for example "Region 'Thiruvallur' is in GI area (Kancheepuram, Thiruvallur)".

Listing 5.2: Node trace showing evidence grading and re-planning (case REPORT-TRACE-1)

```
{"node": "Extract",   "msg": "parsed label text: product=Kanjivaram Silk Saree,
   region=Thiruvallur, state=Tamil Nadu, weaver_id=WV-KAN-001,
   batch_id=B-KAN-0010, material=mulberry silk"}
{"node": "Planner",   "msg": "Initial plan: exact registry match, unfiltered
   document search"}
```

```
{"node": "Retriever", "msg": "gi_registry_lookup X . gi_doc_search OK .
   census_lookup OK . weaver_lookup OK . provenance_lookup OK"}
{"node": "Verifier",  "msg": "5 pass . 0 fail . 5 unknown"}
{"node": "Grader",    "msg": "grade=0.65 (completeness 0.5, consistency 1.0)
   -> re-plan"}
{"node": "Planner",   "msg": "Re-plan #1: resolve aliases -> 'Kancheepuram Silk',
   sharper query filtered by product"}
{"node": "Retriever", "msg": "gi_registry_lookup OK . gi_doc_search OK .
   census_lookup OK . weaver_lookup OK . provenance_lookup OK"}
{"node": "Verifier",  "msg": "10 pass . 0 fail . 0 unknown"}
{"node": "Grader",    "msg": "grade=1.0 (completeness 1.0, consistency 1.0)
   -> finish"}
{"node": "Guardrail", "msg": "Supported"}
```

5.2.4 Observability

Every node appends one entry to the `trace` field, and every tool result is kept in the `evidence` field with its timing in milliseconds. In addition, a wrapper around each node writes the complete state to the `checkpoints` table in SQLite after the node finishes. Listing 5.3 shows the checkpoint rows for the run above: ten rows, one per executed node, with the second planning iteration visible as steps 5–8. Section 7.1 describes what is recorded in more detail. The project's design document proposed LangSmith for tracing; it was not used, because the local trace and checkpoint store provide the same per-node visibility without an external account, and they can be queried with SQL.

Listing 5.3: Checkpoint rows (step, node) stored for case REPORT-TRACE-1

```
(0,'extract') (1,'planner') (2,'retriever') (3,'verifier') (4,'grader')
(5,'planner') (6,'retriever') (7,'verifier') (8,'grader') (9,'guardrail')
```

5.2.5 Evaluation

Quality is measured on a labelled set of 45 cases with exact-match metrics, a single-lookup baseline and per-class precision, recall and F1-score, as reported in Chapter 7. A separate set of 15 hand-written demonstration cases covering nine products is reported in Section 7.5.

5.2.6 Bonus Concepts

Human-in-the-loop and guardrails. Listing 5.4 shows a run in which the label contains no weaver or batch identifier. The verifier marks four producer checks as unknown, the grader finishes because re-planning cannot supply missing identifiers, and the guardrail escalates the case to the review queue with the reason stated. Figure 5.1

shows the reviewer dashboard with such a case awaiting a decision, and Figure 5.2 shows the requester's view after a reviewer confirmed the claim with a note.

Listing 5.4: Trace of an abstention and escalation (case REPORT-TRACE-2)

```
{"node": "Extract",   "msg": "parsed label text: product=Banaras Brocades and
   Sarees, region=Varanasi, state=Uttar Pradesh"}
{"node": "Planner",   "msg": "Initial plan: exact registry match, unfiltered
   document search"}
{"node": "Retriever", "msg": "gi_registry_lookup OK . gi_doc_search OK .
   census_lookup OK"}
{"node": "Verifier",  "msg": "5 pass . 0 fail . 4 unknown"}
{"node": "Grader",    "msg": "grade=0.69 (completeness 0.56, consistency 1.0)
   -> finish"}
{"node": "Guardrail", "msg": "Insufficient evidence -> escalated to human
   (queue #183)"}
reason: "missing: weaver_registered, weaver_cluster, batch_found,
         batch_weaver_match"
```

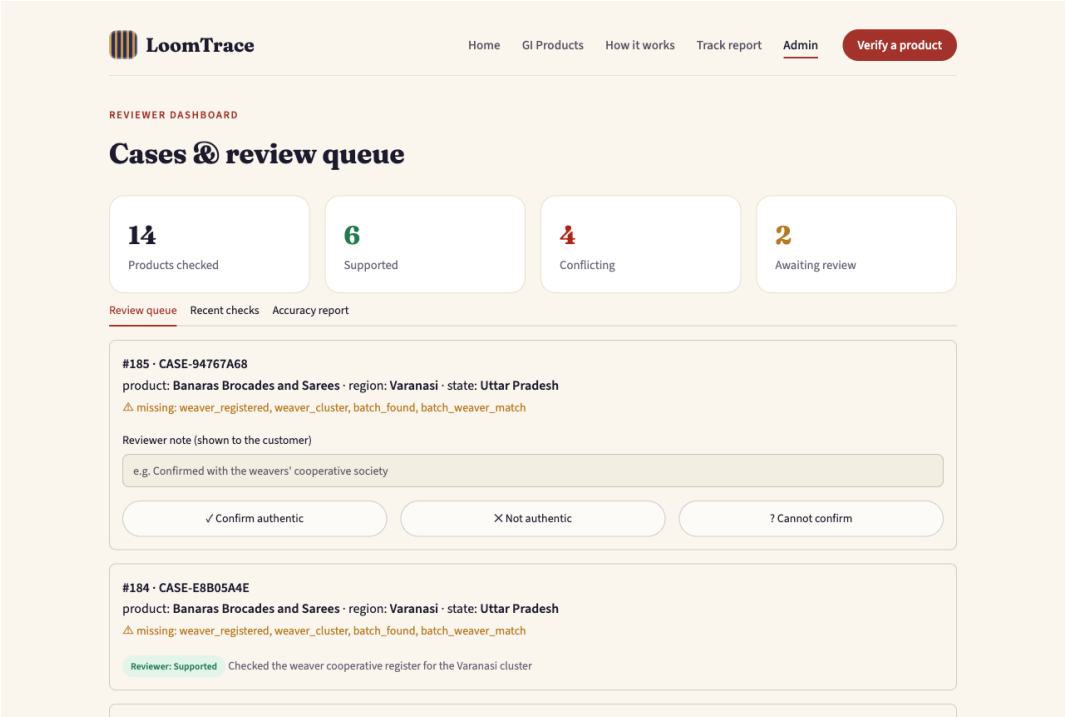


Figure 5.1: Reviewer dashboard: open case with extracted claim, escalation reason, note field and decision buttons; below it, an earlier case already decided by a reviewer

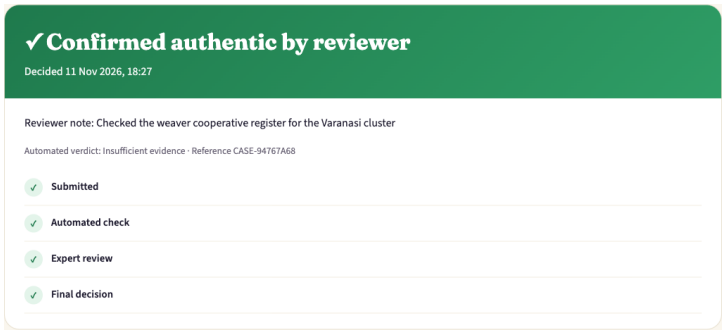


Figure 5.2: Requester's view of an escalated case after the reviewer's decision, with the reviewer's note, the automated verdict and the progress of the case

Multi-agent collaboration (partial). Gathering evidence and judging it are performed by two separate nodes, the retriever agent and the verifier agent, and the graph's conditional routing acts as a supervisor. Both nodes are deterministic in the configuration evaluated, so this is a division of responsibilities rather than a collaboration of independent model-driven agents.

Memory (partial). Each case has its own memory: the CaseState accumulates the evidence and trace of every iteration, the checkpoint table preserves each intermediate state, and the case history and decision tables keep the outcome. There is no memory across cases; for example, the system does not learn that a particular seller's claims are frequently escalated.

Durable workflow (partial). Because the full state is written after every node, a case can be inspected and re-checked at any later time, and human review proceeds asynchronously through the queue. However, the graph does not automatically resume an interrupted run from its last checkpoint; a run that is interrupted is re-executed from the start.

Experimentation. Chapter 8 reports a controlled A/B experiment in which the re-planning loop is disabled while all other factors are held fixed.

CHAPTER 6

IMPLEMENTATION

This chapter describes how the design of Chapter 4 was built: the technology stack, the preparation of the reference data and the retrieval index, the agent loop, the tools, the prompts, the guardrails and loop bounds, the user interface and the repository structure. Together with Appendix B, it is intended to allow another student to reproduce the system.

6.1 Technology Stack

Table 6.1 lists the technologies used, with the exact versions installed in the project's virtual environment.

Table 6.1: Technology stack

Layer	Choice (version)	Reason
Language	Python 3.13.7	Standard for agent tooling
Orchestration	LangGraph 1.2.20 [22]	Explicit state graph with conditional edges and reducers
Schemas	Pydantic 2.14.3 [23]	Typed tool inputs and outputs, validated at run time
Language model (optional)	Llama 3.1 8B Instant [15] via Groq API, client 1.7.0 [25]	Small, fast model with JSON output; free tier
Retrieval	BM25 [11], implemented in Python	Transparent keyword ranking over a small official corpus
OCR (optional)	EasyOCR [26]	Label images; not installed for the reported runs
Persistence	SQLite [27] (Python `sqlite3`)	Checkpoints, review queue, decisions, case history
Interface	Streamlit 1.67.0 [24] with custom HTML and CSS	Web pages for requesters and reviewers
Tables	pandas 3.1.1	Reviewer and evaluation tables

The design document proposed Chroma with `bge-small` embeddings as the vector store. This was replaced by BM25 because the corpus contains only 21 short passages whose relevant terms (district names, materials, product names) are exact keywords,

for which lexical matching is reliable, and because BM25 needs no model download, so the evaluation runs offline and deterministically. Dense retrieval would become preferable if the full scanned GI applications were indexed (Section 10.2).

6.2 Data Preparation and Indexing

Registry. Each of the fifteen products is stored as a record with its GI application number (where known), state, goods class, list of registered districts or places, list of aliases, registered materials, a `state_wide` flag and a source identifier. Aliases were written by the team from common market names, for example "kanjivaram", "kanjeevaram" and "kanchi pattu" for Kancheepuram Silk. When several aliases match a label, the longest match is chosen, so that "Sambalpuri ikat" resolves to Sambalpuri Bandha rather than to Pochampalli Ikat through the shorter alias "ikat".

Census. The 31 rows of the PIB annexure were transcribed into a table of state, weavers and allied workers, keyed by state name. The All-India total in the source (26,73,891 weavers) was used to check the transcription.

Retrieval corpus and index. The corpus has 21 passages: three for each of the three original products and one for each of the twelve catalogue products, the latter taken from the description in the PIB GI catalogue. Each passage carries its product and a page reference as metadata. Documents are not chunked further, because each passage is a single paragraph of 20 to 59 words that describes one aspect of one product; splitting would separate district names from the product they refer to. Text is lower-cased and tokenised into alphabetic words, the product name is appended to each passage, and BM25 statistics (document frequencies and average length) are computed once at start-up. Scoring uses the standard parameters $k_1 = 1.5$ and $b = 0.75$:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \, \frac{f(t, d)\,(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \, \frac{|d|}{\overline{|d|}}\right)}, \tag{6.1}$$

where $f(t, d)$ is the frequency of term t in passage d and $\text{IDF}(t) = \ln\!\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right)$ for N passages of which n_t contain t. The search returns at most five passages with a score above zero, optionally restricted to one product. A first-iteration search is considered adequate evidence only if its top score is at least 2.0. This value was fixed by the team during development as a conservative minimum and was not tuned on the test set; once the product has been resolved through the registry, any passage retrieved for that product is accepted as document support.

Simulated producer records. The generator described in Section 3.4 is run at import time with a fixed seed, producing identifiers of the form `WV-KAN-001` for weavers and `B-KAN-0010` for batches, where the three-letter code identifies the product.

6.3 Core Agent Loop

Algorithm 1 gives the agent loop in pseudocode.

Algorithm 1 LoomTrace agent loop

Require: claim text x, maximum iterations $L = 3$

1: $f \leftarrow \textsc{Extract}(x)$; $i \leftarrow 0$; $E \leftarrow [\,]$
2: **loop**
3: $(P, q, \mathit{resolve}) \leftarrow \textsc{Plan}(f, i)$ ▷ checks, query, alias flag
4: $E \leftarrow E \cup \textsc{Retrieve}(f, q, \mathit{resolve})$ ▷ tool calls, tagged with i
5: $C \leftarrow \textsc{Verify}(f, E_i)$ ▷ pass / fail / unknown per check
6: $g \leftarrow \textsc{Grade}(C)$
7: **if** $\textsc{Unresolved}(C) \wedge f.\mathit{product} \neq \varnothing \wedge i + 1 < L$ **then**
8: $i \leftarrow i + 1$ ▷ re-plan
9: **else**
10: **break**
11: **end if**
12: **end loop**
13: $v \leftarrow$ Conflicting **if** any $c \in C$ fails **else** Insufficient **if** any is unknown **else** Supported
14: **if** $v \neq \textsc{Insufficient} \wedge (\textsc{Citations}(C) = \varnothing \vee g < 0.6)$ **then**
15: $v \leftarrow \textsc{Insufficient}$ ▷ guardrail downgrade
16: **end if**
17: **if** $v = \textsc{Insufficient}$ **then**
18: $\textsc{EscalateToHuman}(x, f, \textsc{Reason}(C))$
19: **end if**
20: **return** v, $\textsc{Citations}(C)$, g

Line 1 extracts the fields and initialises the iteration counter and the evidence list. Lines 3–6 are one planning iteration: the planner chooses the checks and query, the retriever calls the tools and appends their results, the verifier evaluates only the evidence of the current iteration, and the grader computes the grade. Line 7 is the loop condition: the agent re-plans only if the product or the document evidence is still unresolved, a product name is available, and the limit of three iterations has not been reached; otherwise the loop ends at line 10. Line 13 maps the checks to a verdict.

Lines 14–16 apply the guardrail, and lines 17–18 escalate every undecided case. In the implementation, each line of the loop is a separate LangGraph node, the loop condition is a conditional edge, and every node is wrapped by a function that writes a checkpoint after it runs.

6.4 Tools and Function Definitions

Table 6.2 lists the seven tools. Every tool returns a `ToolResult` object with the fields `tool`, `ok`, `data`, `source_id`, `note` and `ms`.

Table 6.2: Tools available to the agent

Tool	Input → output	Data source; error behaviour
`extract_label`	Label text, optional image → `LabelFields`	EasyOCR and SLM if available, else rule parser; never fails, leaves fields empty
`gi_registry_lookup`	Product name, alias flag → GI no., state, districts, materials	Registry (IP India and PIB catalogue); `ok=false` with note if not found
`gi_doc_search`	Query, optional product → top-5 passages with scores and pages	BM25 index of 21 passages; `ok=false` if no passage scores above 0
`census_lookup`	State → weavers, allied workers	Census table (PIB annexure); `ok=false` for unknown state
`weaver_lookup`	Weaver ID → name, cluster, district, looms	**Simulated** registry; `ok=false` for unknown ID
`provenance_lookup`	Batch ID → weaver, loom, yarn, dates	**Simulated** records; `ok=false` for unknown ID
`escalate_to_human`	Case ID, reason, claim → queue entry	SQLite review queue; returns queue number

Two helper functions complete the human-review workflow: `decide`, which records a reviewer's decision and note and closes the queue entry, and `case_status`, which returns the automated verdict, the queue entry and the reviewer's decision for a given case reference.

6.5 Prompt Design

The language model is used for two narrow tasks only, and both calls use temperature 0 and the JSON output mode of the API. The extraction prompt asks for a fixed set of keys, requires `null` for absent fields and requires identifiers to be copied exactly; the

explanation prompt receives only the verdict, the status of each check, and the source identifiers and notes of at most twelve tool results, and is told to use only these and to cite sources in brackets. Both prompts are given in full in Appendix D. Restricting the model to these inputs means that it cannot introduce facts that the tools did not return, and requiring JSON means that malformed output is detected by parsing and replaced by the deterministic fall-back.

All evaluation results in this report were produced *without* an API key, so the model was not called and the deterministic parser and template explanations were used. The prompts have therefore not been evaluated quantitatively, and no claim is made about the model's extraction accuracy.

6.6 Guardrails and Loop Bounds

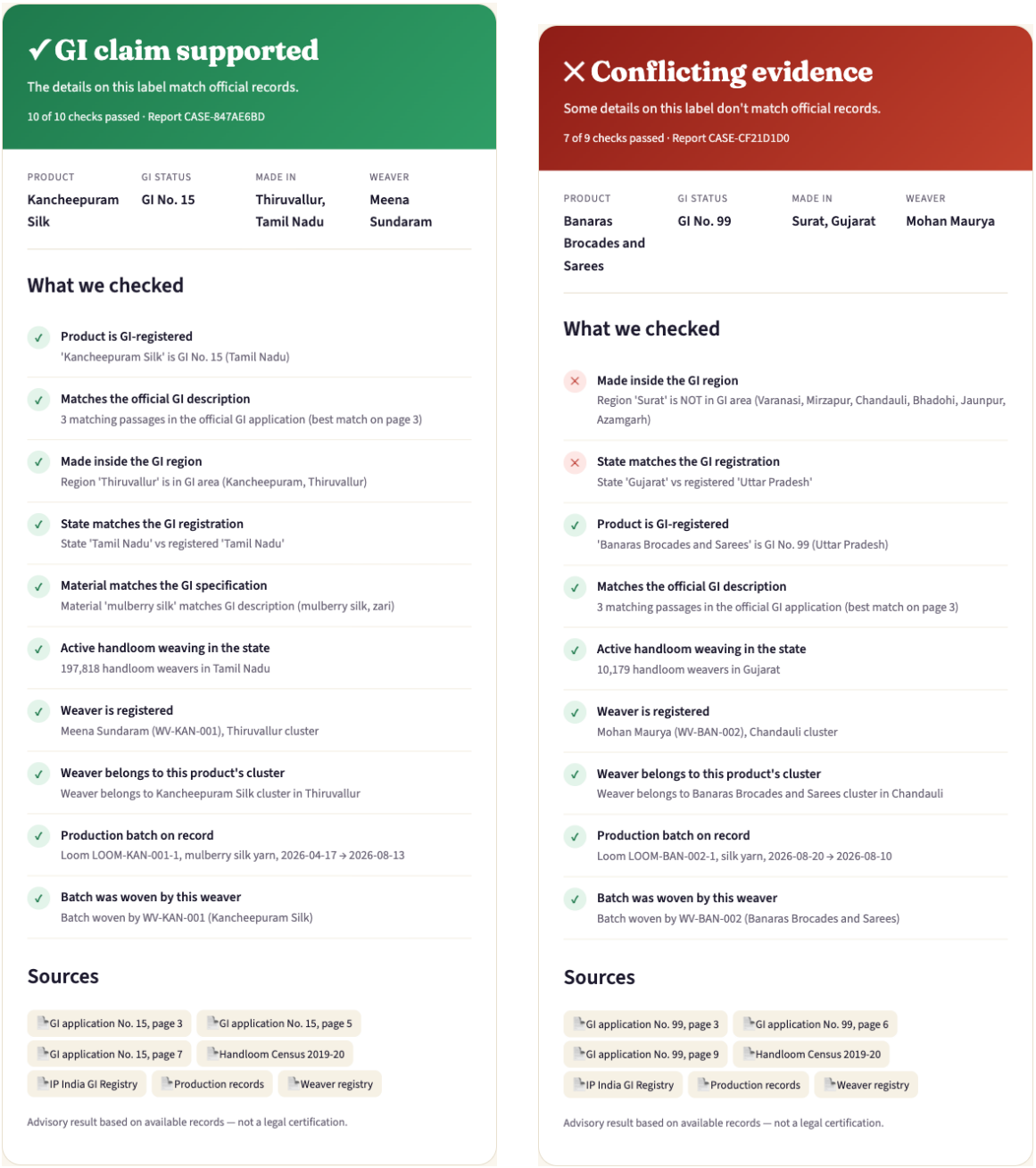
The following guardrails are implemented, with their numerical values.

- **Iteration bound.** At most $L = 3$ planning iterations per case. In the evaluation, the maximum observed was two iterations and eleven tool calls.
- **Graph bound.** LangGraph recursion limit of 40 node executions per run, independent of the iteration counter.
- **Citation and confidence gate.** A *Supported* or *Conflicting* verdict requires at least one citation and a grade of at least 0.6; otherwise it is downgraded to *Insufficient evidence*.
- **Escalation.** Every *Insufficient evidence* verdict is placed in the review queue.
- **Typed input and output.** Extracted fields and tool results are validated by Pydantic models; model output that does not parse as JSON is discarded.
- **No guessing of identity.** An unknown weaver or batch identifier yields *unknown*, not *fail*, so that a typing error does not produce a *Conflicting* verdict.
- **Coverage gate.** A product that does not resolve to one of the fifteen GIs is reported in the interface as "not covered" before the agent runs, with an option to send it to a reviewer.
- **Input handling.** Label text is treated only as data: it is parsed into fields and is never executed or used to select tools.

6.7 User Interface

The interface is a Streamlit application styled as a website with five public pages (Home, GI Products, How it works, Verify a product and Track report) and an Admin

page for reviewers. On the Verify page the requester enters the label fields or chooses an example, and the agent runs while a progress bar shows plain-language status messages. The result is a report card, shown in Fig. 6.1: a coloured verdict header, the resolved product and GI status, the place of manufacture and the weaver, the list of checks with a tick, cross or question mark and a reason for each, and the list of sources. The reviewer dashboard and the tracking page were shown in Figs. 5.1 and 5.2.



(a) Supported (alias resolved) (b) Conflicting evidence (wrong region)

Figure 6.1: Verification reports produced by the interface for two claims

6.8 Repository Structure

Listing 6.1 shows the layout of the repository and the purpose of each main file.

Listing 6.1: Repository layout

```
loomtrace/
|-- README.md              # setup and run instructions
|-- requirements.txt       # streamlit, langgraph, pydantic, pandas, groq
|-- app.py                 # Streamlit website (6 pages)
|-- run_tests.py           # 15 demonstration cases -> TEST_RESULTS.md
|-- loomtrace/
|   |-- agent.py           # CaseState, 6 nodes, graph, guardrail, SLM wrapper,
    baseline
|   |-- tools.py           # 7 tools, BM25 index, SQLite tables, review helpers
|   |-- data.py            # registry (15 GIs), census (31 rows), passages,
|   |                      #   simulated weaver and batch generator
|   |-- evaluate.py        # 45-case golden set and metrics
|   |-- experiments.py     # repeated evaluation and A/B experiment -> results/
|-- results/experiments.json  # raw numbers used in Chapters 7 and 8
|-- loomtrace.db           # SQLite: checkpoints, cases, review_queue, decisions
```

CHAPTER 7

OBSERVABILITY AND EVALUATION

This chapter describes what is recorded for every run and reports how well the agent performs. All numbers come from runs of the scripts `loomtrace/experiments.py` and `run_tests.py` on 11 November 2026 on the hardware described in Section 3.5; the raw output is stored in `results/experiments.json` and `TEST_RESULTS.md`. All runs used the deterministic rule mode, without a language model.

7.1 Tracing and Monitoring

Three levels of information are captured for every case.

1. **Node trace.** Each node appends an entry with its name, a summary message and a timestamp; the planner's entry includes the plan and query, the verifier's includes every check with its status, reason and source, and the grader's includes completeness, consistency, citation fraction and its decision. Listings 5.2 and 5.4 show two such traces.
2. **Tool log.** Each tool call is recorded with its iteration, input-derived source identifier, success flag, note and elapsed time in milliseconds (Listing 5.1).
3. **Checkpoints.** After every node, the complete state is serialised to JSON and stored in the `checkpoints` table with the case identifier, step number and node name (Listing 5.3). The `cases`, `review_queue` and `decisions` tables record the outcome of each case and of each human review.

Because all model calls were disabled, token usage and monetary cost were zero for every run and are not reported further. Latency was measured with a high-resolution timer around each complete graph invocation, including the SQLite writes. Over three repetitions of the 45-case set, the mean latency of the full agent was 16.7 ms per case (standard deviation of the three repetition means 2.9 ms). The slowest single case took 391.5 ms; it was the first case of the first repetition and includes one-time start-up work. In the second and third repetitions, no case took longer than 25 ms. These values are four orders of magnitude below the 15-second requirement (NFR1), as expected for a configuration without model calls.

7.2 Evaluation Dataset

The golden set contains 45 cases: 3 products (Kancheepuram Silk, Pochampalli Ikat, and Banaras Brocades and Sarees) × 3 verdict classes × 5 cases, the structure specified in the design document. Each case is a label text with product, region and state, and, depending on the case, material, weaver identifier and batch identifier. The cases are generated by the script `evaluate.py` from the registry and the simulated producer records according to fixed rules:

- **Supported (5 per product).** A real weaver of the product and one of that weaver's batches, the weaver's district as region, and the batch's yarn as material. Two of the five use a market alias instead of the registered name ("Kanjivaram Silk Saree", "Pochampally Ikat Saree", "Banarasi Silk Saree").
- **Conflicting evidence (5 per product).** One case for each of five contradictions: a region outside the GI area (Surat, Bhagalpur or Murshidabad); a weaver and batch from a different product's cluster; a batch belonging to a different weaver of the same product; a product name that is not a covered GI ("Mysore Silk Saree"); and a material inconsistent with the registration (polyester).
- **Insufficient evidence (5 per product).** Weaver missing; batch missing; both missing; an unknown weaver identifier (`WV-XXX-999`); and an unknown batch identifier (`B-XXX-0000`).

The label of each case is assigned by construction from these definitions, which were written by the team before evaluation; the labels were not assigned independently by two annotators. The class distribution is balanced, 15 cases per class. The test set was not fully separate from development: as described in Section 7.6, one verification rule was changed after the first evaluation run revealed three errors on this set. In addition, a separate set of 15 hand-written demonstration cases (`run_tests.py`) covers nine of the fifteen products, including four products that are not in the golden set (Sambalpuri Bandha, Muga Silk, Kota Doria and Chanderi Sarees) and Kullu Shawls.

7.3 Metrics and Evaluation Rubric

Let $N = 45$ and let y_k and $\hat{y}_k$ be the labelled and predicted verdicts of case k. The following metrics are used.

- **Verdict accuracy** $= \frac{1}{N} \sum_k \mathbb{1}[\hat{y}_k = y_k]$.
- **Per-class precision, recall and F1**, with $F_1 = \frac{2PR}{P+R}$, and **macro-F1**, the unweighted mean of the three class F1-scores.

- **Correct abstention**: the fraction of cases labelled *Insufficient evidence* that are predicted as *Insufficient evidence* and therefore sent to human review.
- **False Supported**: the number of cases predicted *Supported* whose label is not *Supported*. This is the most harmful error, because it endorses an unsupported claim.
- **Retrieval Recall@5**: the fraction of cases in which the final document search returns at least one passage of the correct product among its top five results.
- **Citation rate**: the fraction of decided verdicts (*Supported* or *Conflicting*) that list at least one source identifier.
- **Latency and tool calls**: mean wall-clock time and mean number of tool calls per case.

The design document also proposed *citation faithfulness*, the share of verdict statements backed by the cited text, measured manually. This was not measured: the citation rate above records only that citations are present, not that each cited passage supports the statement. No model was used as a judge, because every output is a discrete label or check status with an exact expected value, for which exact-match metrics are more reliable than a model judge [9].

7.4 Baselines

Two comparison systems were run on the same 45 cases and the same data.

1. **Single-lookup baseline.** This follows the baseline of the design document: one document search using the whole label as the query, followed by one decision. Because no language model was used, the decision step is the rule "*Supported* if the top passage scores above 2, otherwise *Insufficient evidence*". The baseline has no planning, no producer checks, no loop and no guardrail. It is a weak baseline, and it is reported to show what retrieval alone achieves.
2. **LoomTrace without re-planning.** The full agent with the iteration limit set to one, so that the grader can never loop back. This is the variant A of the experiment in Chapter 8 and isolates the contribution of the agentic loop.

7.5 Results

Table 7.1 summarises the results. Because the system and the baseline are deterministic, the three repetitions produced identical verdicts, and the standard deviation of every quality metric is zero; only latency varied between repetitions.

Table 7.1: Evaluation results on the 45-case golden set (mean of 3 repetitions)

Metric	Single lookup	LoomTrace, no re-plan	LoomTrace (full)
Verdict accuracy	33.3%	80.0%	**100.0%**
Macro-F1	0.17	0.80	**1.00**
F1 Supported / Conflicting / Insufficient	0.50 / 0.00 / 0.00	0.75 / 0.89 / 0.77	1.00 / 1.00 / 1.00
Correct abstention (15 cases)	0%	100%	100%
False Supported	30	0	0
Retrieval Recall@5	–	1.00	1.00
Citation rate (decided verdicts)	–	100%	100%
Mean citations per case	–	6.40	6.53
Mean tool calls per case (max)	1 (1)	6.27 (7)	7.07 (11)
Cases that re-planned	–	0	9
Cases escalated to review	–	24	15
Mean latency per case	–	13.7 ms	16.7 ms

As Table 7.1 shows, the single-lookup baseline labels every case *Supported*: the label always contains the product name, so a passage about the product is always retrieved, and the baseline has no means of detecting a contradiction or a missing identifier. It therefore endorses all 30 conflicting and insufficient claims. The full agent classifies all 45 cases correctly, sends exactly the 15 insufficient cases to human review, and never issues a false *Supported* verdict. Nine cases used the re-planning loop: the six *Supported* cases with alias names and the three *Conflicting* cases with an unregistered product name. Table 7.2 gives the confusion matrices.

Table 7.2: Confusion matrices (rows: label; columns: prediction; S = Supported, C = Conflicting, I = Insufficient)

	Single lookup			**No re-plan**			**Full**		
Label	S	C	I	S	C	I	S	C	I
Supported	15	0	0	9	0	6	15	0	0
Conflicting	15	0	0	0	12	3	0	15	0
Insufficient	15	0	0	0	0	15	0	0	15

Figure 7.1 compares the per-class F1-scores of the three systems.

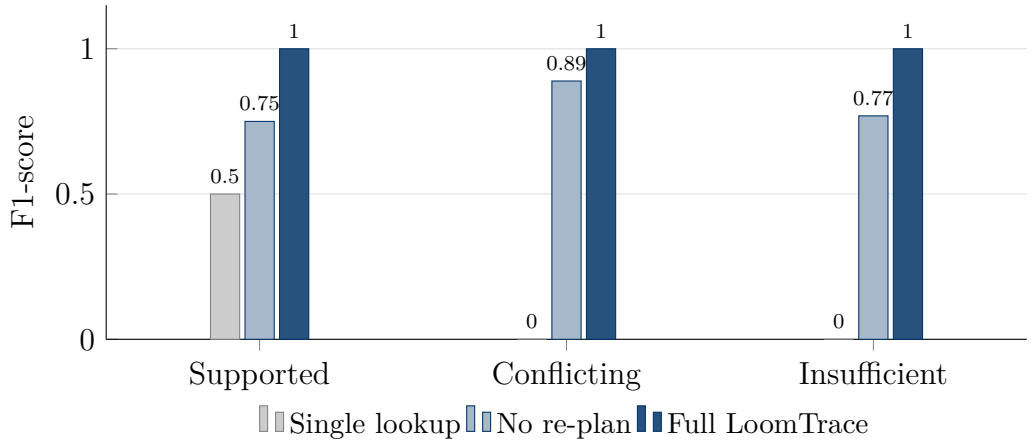


Figure 7.1: Per-class F1-score on the 45-case golden set

Table 7.3 compares the full agent with the targets set in the design document.

Table 7.3: Results against the targets of the design document

Metric	**Target**	**Result**	**Status**
Verdict accuracy	$\geq 80\%$	100%	Met
Retrieval Recall@5	≥ 0.80	1.00	Met
Citation faithfulness	$\geq 90\%$	not measured	Not assessed; citation presence 100%
Correct abstention	$\geq 90\%$	100%	Met
Latency	$< 15\,\text{s}$	16.7 ms	Met (rule mode only)

On the 15 demonstration cases, all 15 verdicts matched the expected verdict: 7 *Supported* (including Sambalpuri Bandha, Muga Silk and Kota Doria), 5 *Conflicting* (wrong region, foreign weaver, batch of another weaver, polyester material, and a Chanderi saree claimed from Varanasi) and 3 *Insufficient* (missing identifiers, unknown weaver, unknown batch). These cases are also team-written and therefore

provide functional coverage of the additional products rather than an independent measure of accuracy.

7.6 Error Analysis

The full agent made no errors on the final golden set. This result must be read with the construction of the test set in mind, and the analysis below therefore covers four sources of errors: errors observed during development, errors of the ablated agent and the baseline, and errors on realistic inputs that the golden set does not contain.

Errors found and corrected during development. The first evaluation run of the full agent reached 93.3% accuracy (42 of 45) and 80% correct abstention. All three errors were cases with an unknown weaver identifier and a valid batch, labelled *Insufficient* but predicted *Conflicting*, because the batch's weaver did not match the unknown identifier. The rule was changed so that a batch–weaver mismatch is reported as *unknown* when the claimed weaver itself cannot be verified; after this change all 45 cases were correct. Because the change was made after observing the test set, the final 100% is partly a result of fitting to it. Separately, the demonstration tests revealed that a claimed material of "silk" was rejected for Kancheepuram Silk, whose registered material is "mulberry silk"; the material comparison was changed to accept a match in either direction.

Errors without re-planning. All nine errors of the ablated agent fall into two categories: six alias names that were not resolved by exact matching (*Supported* predicted as *Insufficient*) and three unregistered product names that could not be classified as *Conflicting* on the first iteration. These are exactly the nine cases that the full agent resolves by re-planning, which is analysed in Chapter 8.

Errors of the baseline. All 30 errors are false *Supported* verdicts, since the baseline never checks region, state, material or producer records.

Errors on realistic inputs outside the test set. To estimate behaviour beyond the constructed cases, six further inputs that vary the spelling and form of the label were run through the full agent. Table 7.4 shows the results. Four of the six produced a wrong *Conflicting* verdict.

Table 7.4: Behaviour on six realistic inputs not in the golden set

Input variation	**Verdict**	**Cause**
Alias not in list (“Conjeevaram Silk Saree”)	Conflicting (wrong)	Product unresolved after re-plan, so the product check fails
District spelling (“Kanchipuram”)	Conflicting (wrong)	Region compared by substring; spelling variant not matched
State abbreviation (“TS”)	Conflicting (wrong)	State compared by exact name; census lookup also fails
Former state name (“Andhra Pradesh” for Pochampalli)	Conflicting (wrong)	Registry stores current state (Telangana) only
Lower-case identifiers	Supported (correct)	Identifiers are upper-cased before lookup
Free-text sentence with alias and identifiers	Supported (correct)	Parser finds product alias and identifier patterns

These four failures share one cause: the region, state and product comparisons are exact or substring matches against a single canonical spelling. They would be fixed by (i) fuzzy matching of district and product names against the registry, with a similarity threshold below which the check is *unknown* rather than *fail*; (ii) a table of state abbreviations and former state names; and (iii) treating a product that cannot be resolved as “not covered” inside the agent, as the interface already does before the agent runs. The more general lesson is that a contradiction should only be reported when the mismatch is certain, and that uncertain mismatches should be routed to human review.

CHAPTER 8

EXPERIMENTATION

This chapter reports one controlled A/B experiment. The factor varied is the re-planning loop, which is the main agentic element of LoomTrace: in variant A the grader may never send control back to the planner, and in variant B, the default configuration, it may do so up to two times. The experiment answers whether this loop is worth its cost.

8.1 Hypothesis

Allowing the agent to re-plan and retrieve again when the first registry lookup or document search is inconclusive (variant B) will increase verdict accuracy compared with a single planning pass (variant A), without increasing false *Supported* verdicts, at a latency cost of less than one second per case.

The hypothesis is falsifiable in three ways: accuracy could fail to rise, the loop could introduce false *Supported* verdicts by resolving names too permissively, or the extra iteration could cost more than one second.

8.2 Experimental Setup

Variants. Variant A sets the maximum number of planning iterations to one. Variant B sets it to three, the value used in all other chapters. In both variants, the first iteration is identical: an exact registry match and an unfiltered document search. Only variant B can perform the second iteration, which resolves aliases and filters the search to the resolved product.

Fixed factors. The code, the graph, the tools, the reference data, the BM25 parameters, the grading formula, the guardrail thresholds and the 45-case golden set were identical for both variants. No language model was used in either variant.

Procedure. The script `loomtrace/experiments.py` ran both variants on all 45 cases, alternating A and B within each of three repetitions, so that any warm-up effect affects both variants. For each case it recorded the verdict, the number of iterations, the number of tool calls, the number of node executions and the wall-clock time.

Metrics. Verdict accuracy, macro-F1, false *Supported* verdicts and correct abstention, as defined in Section 7.3; mean tool calls and node executions per case; and mean latency per case, reported as the mean and standard deviation of the three repetition means.

8.3 Results

Table 8.1 gives the results, and Figure 8.1 shows accuracy and mean tool calls for the two variants.

Table 8.1: A/B experiment: single planning pass (A) versus re-planning loop (B), 45 cases, 3 repetitions

Metric	Variant A	Variant B	Difference (B − A)
Verdict accuracy (mean ± s.d.)	80.0% ± 0.0	100.0% ± 0.0	+20.0 points
Macro-F1	0.80	1.00	+0.20
Correct abstention	100%	100%	0
False Supported	0	0	0
Cases escalated to review	24	15	−9
Mean tool calls per case	6.27	7.07	+0.80
Maximum tool calls per case	7	11	+4
Mean node executions per case	6.00	6.80	+0.80
Mean latency per case (mean ± s.d.)	13.7 ± 1.7 ms	16.7 ± 2.9 ms	+3.1 ms

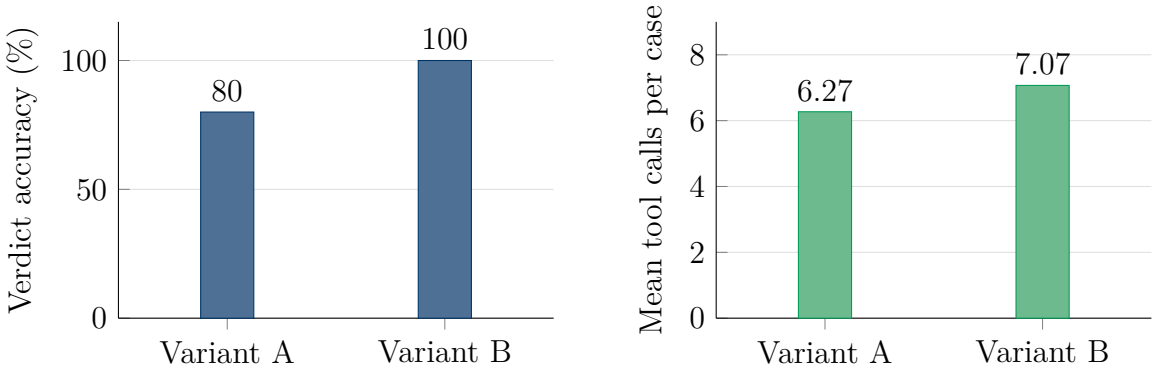


Figure 8.1: Verdict accuracy (left) and mean tool calls per case (right) for the two variants

The accuracy of both variants was identical in all three repetitions, because the system is deterministic; the variation between repetitions is confined to latency. Per-repetition mean latencies were 11.3, 14.4 and 15.4 ms for variant A and 20.7, 13.8 and 15.8 ms for variant B. The first repetition of variant B includes one case of 391.5 ms caused by start-up work, which raises that repetition's mean; the difference in mean

latency between the variants is therefore of the same order as the variation between repetitions.

All nine cases on which the variants differ are cases in which the first iteration could not resolve the product: in variant A they end as *Insufficient evidence*, six of which should have been *Supported* and three *Conflicting*. Variant B re-plans on exactly these nine cases and on no others; each re-planned case executes ten nodes and eleven tool calls instead of six nodes and five to seven tool calls.

8.4 Discussion and Decision

The hypothesis was supported on all three counts. The re-planning loop increased verdict accuracy by 20 percentage points, from 80.0% to 100.0%, and macro-F1 from 0.80 to 1.00. It introduced no false *Supported* verdicts, because the second iteration only resolves names that match a registered alias and still requires every check to pass. Its cost was 0.8 additional tool calls per case on average and an increase in mean latency of about 3 ms, far below the one-second bound and comparable to the run-to-run variation.

The experiment also shows where the benefit comes from. Variant A is not unsafe: it never endorses a false claim and it abstains on all unclear cases. Its weakness is that it abstains too often, sending 24 cases to human review instead of the 15 that genuinely require it. The loop therefore reduces the workload of human reviewers by nine cases out of 45 (20%) without lowering safety. In a deployment where reviewers are the scarce resource, this is the most important effect.

Two cautions apply. First, the result was obtained on a test set in which six of 45 cases deliberately use aliases; the size of the benefit depends on how often real labels use non-registered names. Second, Chapter 7 showed that names outside the alias list are not resolved even with re-planning, so the loop helps only as far as the alias table and matching rules allow. **Decision:** variant B, with a maximum of three planning iterations, was adopted as the default configuration, because it improves accuracy and reduces reviewer workload at negligible cost and with no loss of safety.

CHAPTER 9

LIMITATIONS, ETHICS AND SAFETY

This chapter states the limitations of the data, the method and the evaluation, lists the ways in which the agent can fail together with the guardrail that addresses each, and discusses the ethical considerations of a system whose output affects producers and buyers.

9.1 Limitations

Data. The producer-level evidence, the weaver registry and the batch records, is simulated. Real individual records are not published, so the weaver and batch checks demonstrate the mechanism but say nothing about the availability or quality of real records. The GI description passages for the three original products were written by the team as summaries and their page references are indicative; they were not extracted from the scanned GI application documents. For the twelve products taken from the government GI catalogue, the descriptions are short and the catalogue does not give GI application numbers. The registry stores one current state per product, so historical state names, such as Andhra Pradesh for products now in Telangana, are not recognised.

Method. All factual checks are deterministic string comparisons. As shown in Table 7.4, spelling variants of districts, state abbreviations and aliases that are not in the alias table lead to wrong *Conflicting* verdicts. Retrieval uses BM25 over 21 short passages, which is adequate for this corpus but would not scale to the full text of GI applications without dense retrieval and chunking. The language-model path for extraction and explanation is implemented but was not evaluated, and the OCR path for label images was not installed or tested. The durable-workflow support stores every state but does not resume interrupted runs automatically.

Evaluation. The 45-case golden set and the 15 demonstration cases were constructed by the team from the same rules and records that the agent uses, the labels were assigned by construction rather than by independent annotators, and one rule was changed after the first evaluation run. The 100% accuracy therefore shows that the agent behaves as designed on controlled cases; it is not an estimate of accuracy on real marketplace listings. Citation faithfulness was not measured. The golden set covers three of the fifteen products. The single-lookup baseline is weak by design.

9.2 Failure Modes and Guardrails

Table 9.1 pairs each identified failure mode with its mitigation.

Table 9.1: Failure modes and mitigations

Failure mode	**Mitigation**
Verdict issued without evidence	Citation gate: a decided verdict needs at least one source ID and a grade ≥ 0.6, otherwise it is downgraded to *Insufficient* and escalated
Weak or off-product retrieval	Grader detects unresolved product or weak search; planner re-queries with canonical name and product filter; maximum three iterations
Unbounded looping or tool calls	Iteration limit of 3 and a graph recursion limit of 40; observed maximum 2 iterations and 11 tool calls
Missing weaver or batch details	Checks marked *unknown*, never *pass*; case escalated to review with the missing items listed
Typing error in an identifier	Unknown identifier gives *unknown*, not *fail*; batch mismatch is not asserted against an unverified weaver
Spelling variants causing false conflicts (observed, Table 7.4)	Not yet mitigated in the agent; planned fuzzy matching with an *unknown* band, and abbreviation and former-name tables (Section 10.2)
Product outside the covered GIs	Interface reports "not covered" before the agent runs and offers referral to a reviewer
Model returns malformed or invented fields	JSON mode, Pydantic validation and deterministic fall-back parser; the model never performs factual checks
Instructions embedded in label text (prompt injection)	Label text is only parsed into fields; tools are fixed by the graph, not chosen by the model; the explanation prompt receives only check statuses, source IDs and tool notes
Reviewer error or bias	Reviewer must record a note; the decision is stored with the automated verdict, so both remain auditable

9.3 Ethical and Responsible-Use Considerations

Who could be harmed. A wrong *Conflicting* verdict on a genuine product could damage the reputation and sales of a weaver who is legitimately entitled to the GI, and Chapter 7 showed that spelling variants can produce exactly this error. A wrong *Supported* verdict would mislead a buyer and undermine the value of the GI for all producers in the region. The design therefore treats a false *Supported* verdict as the most serious error, and none occurred in the evaluation; the observed false *Conflicting* verdicts on realistic spelling variants are the most important weakness to correct before any real use.

Advisory status. Every verdict is labelled in the interface as "advisory result based on available records — not a legal certification". Authority over GI registration and authorised use rests with the GI Registry under the GI Act [16], and the system does not claim to replace it. Every verdict shows the checks performed and the sources used, so that a user can see why it was reached, and any user can send a decided case to a human reviewer.

Human oversight. Cases that the agent cannot decide are never resolved automatically. They are routed to a reviewer, whose decision and note are stored separately from the automated verdict and shown to the requester. This keeps a person accountable for every decision that is not fully supported by evidence.

Privacy. Weaver records can contain personal information. In this project, all weaver records are simulated, and the names are randomly combined from lists of common first names and surnames; they do not refer to real people. A real deployment would require consent from weavers and cooperatives, access control on the weaver registry and the review dashboard, and a retention policy for case data, which are not implemented here; the current Admin page has no authentication.

Fairness. Coverage is uneven: fifteen GIs are supported, the alias table reflects the names known to the team, and products from regions with less documentation are more likely to be marked *Insufficient* or "not covered". This should be reduced by extending coverage with producer associations rather than by relaxing the checks.

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 Conclusion

This report addressed the problem of checking whether a GI-tag claim on an Indian handloom product is supported by official evidence, a task for which the necessary information is spread across the GI registry, GI descriptions, the national handloom census and producer records. It presented LoomTrace, an agent implemented as a six-node LangGraph state graph with seven typed tools, retrieval over official GI text, a bounded re-planning loop, an evidence grader, a citation and confidence guardrail, and a human review workflow, covering fifteen registered handloom GIs.

The objectives stated in Section 1.4 were addressed as follows.

1. **Agent design.** Met. The state-graph agent returns one of three verdicts with the result and source of every check (Chapter 4).
2. **Tools, retrieval and bounded re-planning.** Met. Seven typed tools and a BM25 index over 21 passages were implemented; the loop is limited to three iterations and in practice used at most two (Chapter 6).
3. **Guardrail and human review.** Met. All 15 unclear cases in the test set were escalated, and reviewer decisions with notes are recorded and shown to the requester (Chapters 5 and 7).
4. **Tracing and checkpoints.** Met. Every node writes a trace entry and a checkpoint; LangSmith, proposed in the design document, was replaced by this local store (Chapter 5).
5. **Evaluation against a baseline.** Met for accuracy and abstention. On the 45-case golden set the agent reached 100% accuracy and 100% correct abstention against targets of 80% and 90%, while a single-lookup baseline reached 33.3% and 0%. Citation faithfulness was not measured, and the result holds for team-constructed cases only; on six realistic spelling variants the agent produced four wrong *Conflicting* verdicts (Chapter 7).
6. **A/B experiment.** Met. Disabling re-planning reduced accuracy from 100% to 80% and increased escalations from 15 to 24, while the loop cost 0.8 tool calls and about 3 ms per case; re-planning was adopted as the default (Chapter 8).

The main conclusion is that, for this verification task, the agentic elements contribute through control rather than through generation: re-planning recovers cases that a single pass would wrongly leave undecided, and the guardrail ensures that undecidable cases reach a person instead of receiving a guessed verdict.

10.2 Future Work

The following extensions follow directly from the limitations in Chapter 9.

1. **Robust name matching.** Add fuzzy matching of product, district and state names with a similarity band in which checks are marked *unknown* rather than *fail*, together with tables of state abbreviations and former state names, so that the four failures in Table 7.4 become correct or escalated verdicts.
2. **Official GI application text.** Download and OCR the statements of case of the covered GIs from the IP India registry, index them with chunking and dense retrieval, and record the GI application numbers of all fifteen products, replacing the team-written summaries and indicative page numbers.
3. **Real producer data.** Replace the simulated weaver and batch records with records from a partner weavers' cooperative, with consent and access control, and evaluate on real labels.
4. **Independent evaluation.** Build a test set from real marketplace listings labelled by two independent annotators, measure inter-annotator agreement and citation faithfulness, and evaluate the language-model extraction path on it.
5. **Automatic resumption.** Use the stored checkpoints to resume interrupted runs and to pause the graph at the review step, so that a reviewer's decision continues the same run.

BIBLIOGRAPHY

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[3] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[6] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[7] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[8] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024.

[9] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[10] L. Wang *et al.*, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[11] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[12] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[13] N. F. Liu *et al.*, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, 2024.

[14] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[15] A. Grattafiori *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[16] Government of India, "The Geographical Indications of Goods (Registration and Protection) Act, 1999," Act No. 48 of 1999, Ministry of Law, Justice and Company Affairs, 1999.

[17] World Trade Organization, "Agreement on Trade-Related Aspects of Intellectual Property Rights," Annex 1C of the Marrakesh Agreement, Part II, Section 3 (Articles 22–24), 1994.

[18] Office of the Controller General of Patents, Designs and Trade Marks, "Geographical Indications Registry," Intellectual Property India. [Online]. Available: https://ipindia.gov.in. Accessed: Nov. 2026.

[19] Ministry of Textiles, Government of India, "State/UT-wise number of handloom weavers and allied workers as per 4th All India Handloom Census 2019–20," Annexure to Press Information Bureau release ID 2202869, Dec. 2025. [Online]. Available: https://static.pib.gov.in/WriteReadData/specificdocs/documents/2025/dec/doc20251212729201.pdf. Accessed: Nov. 2026.

[20] Tribal Co-operative Marketing Development Federation of India (TRIFED), Ministry of Tribal Affairs, "Geographical Indications of India," Press Information Bureau document, Nov. 2021. [Online]. Available: https://static.pib.gov.in/WriteReadData/specificdocs/documents/2021/nov/doc2021112441.pdf. Accessed: Nov. 2026.

[21] Office of the Development Commissioner (Handlooms), Ministry of Textiles, "Handloom Mark and India Handloom Brand schemes." [Online]. Available: https://handlooms.nic.in. Accessed: Nov. 2026.

[22] LangChain Inc., "LangGraph documentation," 2025. [Online]. Available: https://langchain-ai.github.io/langgraph/. Accessed: Nov. 2026.

[23] S. Colvin *et al.*, "Pydantic: Data validation using Python type hints," 2025. [Online]. Available: https://docs.pydantic.dev. Accessed: Nov. 2026.

[24] Snowflake Inc., "Streamlit documentation," 2025. [Online]. Available: https://docs.streamlit.io. Accessed: Nov. 2026.

[25] Groq Inc., "GroqCloud API documentation: supported models," 2025. [Online]. Available: https://console.groq.com/docs/models. Accessed: Nov. 2026.

[26] JaidedAI, "EasyOCR: Ready-to-use OCR with 80+ supported languages," GitHub repository. [Online]. Available: https://github.com/JaidedAI/EasyOCR. Accessed: Nov. 2026.

[27] D. R. Hipp, "SQLite," 2025. [Online]. Available: https://www.sqlite.org. Accessed: Nov. 2026.

CHAPTER A

TEAM CONTRIBUTION

Table A.1 lists the components owned by each member, following the roles assigned in the project's design document. All members took part in the reviews and are able to explain the complete system.

Table A.1: Contribution of team members

USN	Name	Components owned	Share
1BM23CD027	Sudeeksha (Lead)	Project coordination, problem charter, design document, documentation and report	25%
1BM23CD026	Khushi Vernekar	Agent and RAG implementation: graph, CaseState, planner, grader, re-planning loop, guardrail, retrieval index, prompts	25%
1BM23CD066	Trishul J	Data collection (GI registry, PIB catalogue, census), simulated records, tools, web interface	25%
1BM24CD400	Geetha R	Golden test set, metrics, baseline, A/B experiment, demonstration tests, error analysis	25%

CHAPTER B

SETUP AND RUN INSTRUCTIONS

The source code, the evaluation results and the source of this report are available at https://github.com/hrterrain/loomtrace-agent. The system runs on any computer with Python 3.11 or later; no graphics processor or internet connection is needed in rule mode. The commands below create the environment, start the interface and reproduce every number in Chapters 7 and 8.

```
git clone https://github.com/hrterrain/loomtrace-agent.git
cd loomtrace-agent
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt          # streamlit, langgraph, pydantic, pandas,
    groq

# optional: enable Llama 3.1 8B on Groq for extraction and explanations
export GROQ_API_KEY=<your key>          # never commit the key

# optional: label images
pip install easyocr

streamlit run app.py                     # web interface on http://localhost:8501
python -m loomtrace.experiments          # Tables 7.1-7.3 and 8.1 -> results/
    experiments.json
python -m loomtrace.evaluate             # single 45-case evaluation
python run_tests.py                      # 15 demonstration cases -> TEST_RESULTS.
    md
```

Versions used for the reported runs: Python 3.13.7, LangGraph 1.2.20, langchain-core 1.7.7, Pydantic 2.14.3, Streamlit 1.67.0, pandas 3.1.1, groq 1.7.0, on macOS 15.6 (Apple M2, 8 GB). The SQLite database `loomtrace.db` is created automatically on first run.

CHAPTER C

SAMPLE TRACES AND SCREENSHOTS

This appendix gives three complete runs: a successful run, a run that required a correction through re-planning, and an abstention. All were produced by the system on 11 November 2026.

C.1 Successful Run with Correction: Kanjivaram Silk Saree

The node trace of this run is Listing 5.2, its tool log is Listing 5.1, and its checkpoints are Listing 5.3. The first iteration could not resolve the alias "Kanjivaram" (grade 0.65); the second resolved it to Kancheepuram Silk, all ten checks passed (grade 1.0), and the verdict *Supported* was issued with seven citations: `GI15-p3`, `GI15-p5`, `GI15-p7`, `HLCENSUS-2019-20#Tamil_Nadu`, `IPINDIA-GI-REG#15`, `SIM-PROV#B-KAN-0010` and `SIM-WEAVER#WV-KAN-001`. The report shown to the requester is Fig. 6.1(a).

C.2 Contradiction: Banarasi Saree Claimed from Surat

Listing C.1 shows a claim for a Banarasi silk saree said to be made in Surat, Gujarat, with a valid Banaras weaver and batch. The alias required one re-plan; the verifier then found two failed checks, and the grader's consistency score of 0 led to *Conflicting evidence*. The report is Fig. 6.1(b).

Listing C.1: Trace of a conflicting claim (case REPORT-TRACE-3)

```
{"node": "Extract",   "msg": "parsed label text: product=Banarasi Silk Saree,
   region=Surat, state=Gujarat, weaver_id=WV-BAN-002, batch_id=B-BAN-0020"}
{"node": "Planner",   "msg": "Initial plan: exact registry match, unfiltered
   document search"}
{"node": "Retriever", "msg": "gi_registry_lookup X . gi_doc_search OK .
   census_lookup OK . weaver_lookup OK . provenance_lookup OK"}
{"node": "Verifier",  "msg": "5 pass . 0 fail . 4 unknown"}
{"node": "Grader",    "msg": "grade=0.69 (completeness 0.56, consistency 1.0)
   -> re-plan"}
{"node": "Planner",   "msg": "Re-plan #1: resolve aliases -> 'Banaras Brocades
   and Sarees', sharper query filtered by product"}
{"node": "Retriever", "msg": "gi_registry_lookup OK . gi_doc_search OK .
   census_lookup OK . weaver_lookup OK . provenance_lookup OK"}
{"node": "Verifier",  "msg": "7 pass . 2 fail . 0 unknown"}
```

```
{"node": "Grader",    "msg": "grade=1.0 (completeness 1.0, consistency 0.0)
   -> finish"}
{"node": "Guardrail", "msg": "Conflicting evidence"}
rationale: "Conflict found: Region 'Surat' is NOT in GI area (Varanasi,
  Mirzapur, Chandauli, Bhadohi, Jaunpur, Azamgarh); State 'Gujarat' vs
  registered 'Uttar Pradesh'."
```

C.3 Abstention: Banaras Saree without Producer Details

The trace of this run is Listing 5.4. Figure C.1 shows the report presented to the requester, which states that the case has been sent to an expert reviewer and lists the four producer checks that could not be made. The reviewer's dashboard and the requester's tracking view for such a case are Figs. 5.1 and 5.2.

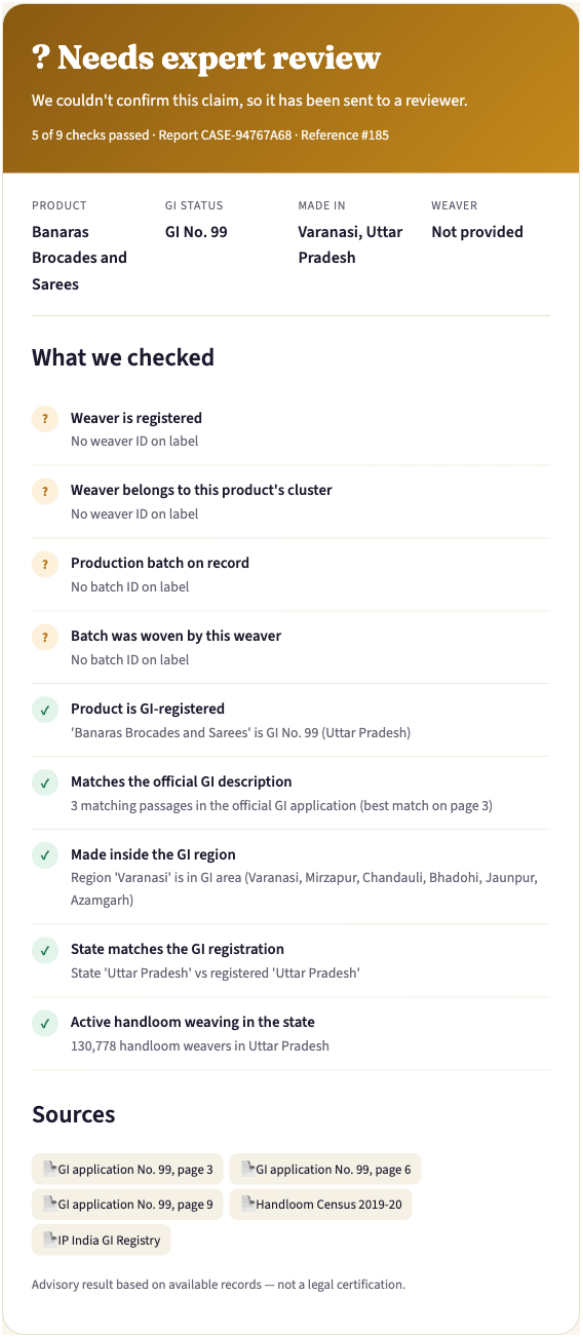


Figure C.1: Report for a claim without weaver or batch details, escalated to human review

CHAPTER D

PROMPTS

LoomTrace uses a language model only in the optional model mode, for two tasks. Both prompts are sent as the system message to `llama-3.1-8b-instant` on Groq with temperature 0 and `response_format={"type": "json_object"}`. As stated in Section 6.5, these prompts were not used in the reported evaluation. No model is used as a judge, so there is no judge rubric.

Listing D.1: System prompt for field extraction; the user message is the label text

```
Extract handloom label fields. Return JSON with keys: product, region (district),
state, weaver_id, batch_id, material, gi_claim (bool). Use null if absent.
Copy IDs exactly.
```

Listing D.2: System prompt for the verdict explanation; the user message is a JSON object with the verdict, the status of each check and up to twelve source IDs with notes

```
You explain a GI verification verdict in 2-3 sentences. Only use the given checks
and cite sources in [brackets]. Return JSON {"rationale": str}.
```

The extraction prompt asks for a fixed key set so that the output can be validated by the `LabelFields` model, requires `null` for absent fields so that missing information is not invented, and requires identifiers to be copied exactly because they are used as database keys. The explanation prompt limits the model to the checks already computed, so that the explanation cannot introduce facts that were not verified. If either call fails or returns invalid JSON, the deterministic parser or the template explanation is used instead.